Red House: documento técnico del segundo parcial

Proyecto 6, equipo 01. Entrega prevista: 22 de octubre de 2026. Edición de implementación 0.3.0. Las referencias a evidencia apuntan a archivos ejecutados y conservados; los ejemplos de contratos se identifican por separado.

Actualización del 20 de octubre: las sesiones técnicas del portal usan Redis y las revocaciones pendientes PostgreSQL. Las mediciones integrales del 19 de octubre se conservan como antecedentes; la nueva validación de sesiones documenta esta sustitución, su migración y sus pruebas.

1. Resumen ejecutivo y problema

Red House coordina una red regional de bancos de sangre e instituciones participantes. Una institución registra recursos disponibles y otra solicita su traslado; el sistema conserva quién registró, evaluó, autorizó, reservó, transportó y recibió cada recurso. El problema operativo es la dispersión de información, las reservas concurrentes, la pérdida de trazabilidad y la dificultad para distinguir disponibilidad real de información desactualizada.

La entrega integra un portal Flask/Jinja2, diez servicios de dominio y un servicio de Monitoreo. PostgreSQL conserva la información transaccional en una base compartida con permisos separados. Redis participa en cada petición protegida. MongoDB conserva eventos, telemetría y proyecciones documentales. Google Cloud Storage almacena evidencias privadas. Docker proporciona un entorno separado del entorno habitual, con dos instancias por API y un gateway.

El portal incorpora su propio backend modular. Ejecuta los mismos módulos versionados de aplicación que se empaquetan en los servicios, con las mismas validaciones, autorización y persistencia. Por ello puede ejecutar operaciones cuando las API están detenidas. Los futuros clientes Android y escritorio utilizarán exclusivamente las API con JSON y XML, respectivamente; su implementación está fuera de este parcial.

Este es un sistema académico de coordinación y apoyo. La elegibilidad, compatibilidad clínica, conservación, asignación y trasplante requieren valoración y autorización profesional. Un resultado del algoritmo no constituye autorización clínica. Los datos de las pruebas son ficticios.

2. Objetivos, alcance y actores

El objetivo general es ejecutar y demostrar procesos regionales trazables, con seguridad común, persistencia real y continuidad ante la caída de una instancia de aplicación.

Los objetivos específicos son conservar registros de donantes y receptores; controlar inventario y solicitudes; obtener candidatos explicables; registrar revisión humana; impedir reservas duplicadas; coordinar traslado y recepción; conservar custodia, notificaciones y auditoría; demostrar contratos JSON/XML; y reproducir el despliegue con contenedores y secretos externos.

Actor	Responsabilidad implementada	Restricción principal
Administrador	Usuarios, instituciones y catálogos autorizados	Administrar cuentas no concede acceso clínico general
Médico	Receptores, solicitudes, dictámenes, liberación y reserva	Ámbito institucional, versión de registro y confirmación humana
Operador de banco	Registro de recursos, donaciones, preparación y recepción	Institución participante en el recurso o traslado
Coordinador regional	Rutas, planeación, seguimiento y panel regional	Región asignada; no sustituye dictamen médico
Transportista	Recolección, traslado, entrega y ubicación	Solo traslados asignados y estados autorizados
Auditor	Historial minimizado y monitoreo	Región y perfil; no modifica eventos
Donante	Registro preliminar, campañas, citas y avisos propios	Acceso a su cuenta y expediente autorizado

La implementación distingue sangre y órganos mediante tipos, componentes, estados y reglas de evaluación. No interpreta HLA ni decide aptitud clínica. No implementa Android ni escritorio. La recuperación automatizada de contraseña, email/push, agenda detallada por sede/hora y reportes regulatorios certificados continúan pendientes. La ayuda de acceso y la exportación CSV de una página no se presentan como esas funciones.

3. Correcciones y trazabilidad

Las correcciones siguientes se comprueban en código y pruebas. No se atribuyen al docente observaciones que el equipo no proporcionó.

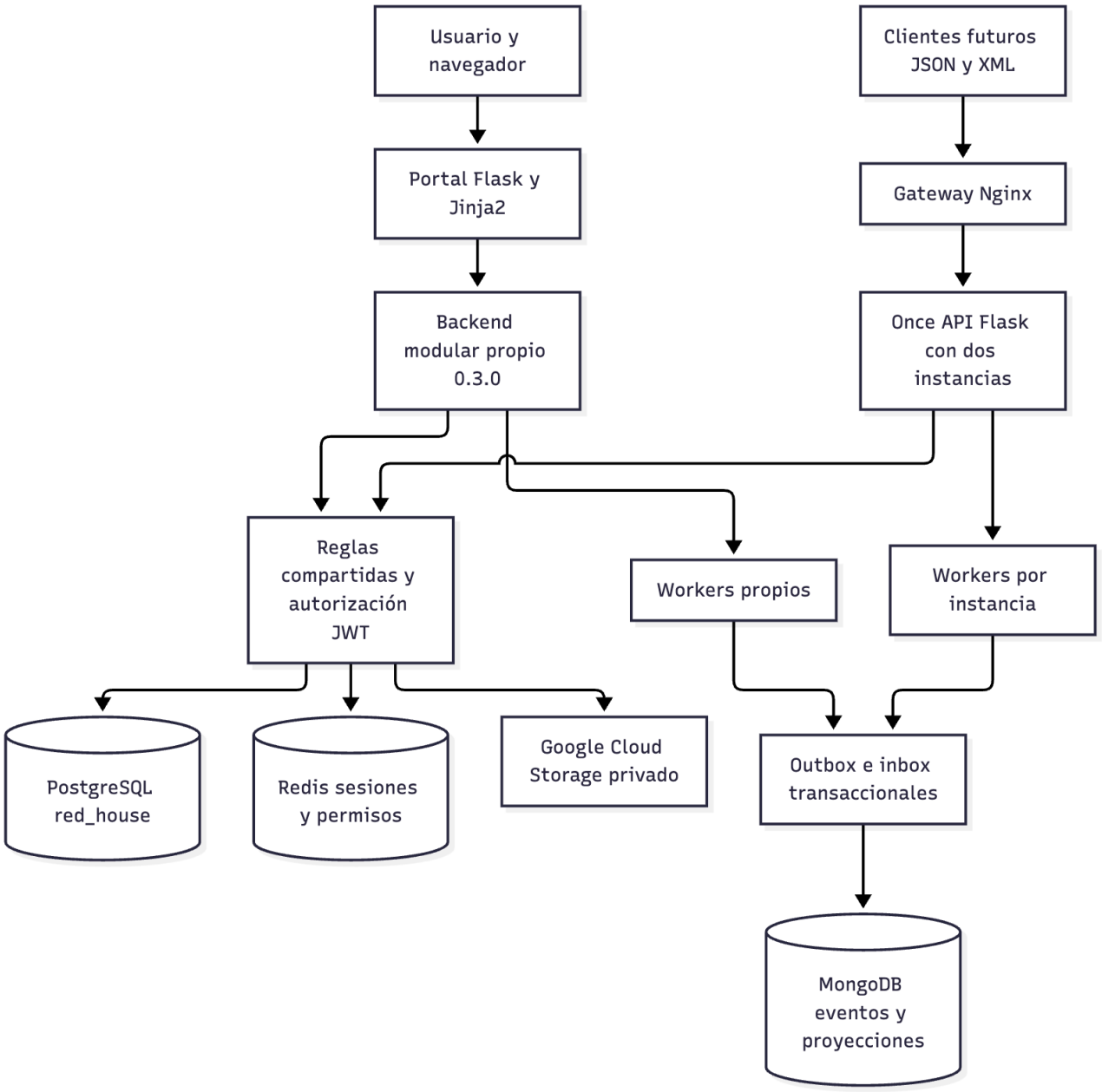
Situación anterior	Cambio del segundo parcial	Criterio verificable
Portal dependiente de HTTP para todo el negocio	Backend modular propio seleccionado al configurar el portal	Sangre y órganos con todas las API detenidas
Una instancia por API	Dos instancias de cada propietario, incluida Identidad, con workers	JSON/XML y flujo completo con las instancias A detenidas
Persistencia separada por servicio en antecedentes	Base red_house, 71 tablas y vistas access_*	Esquema físico, permisos y pruebas de la base compartida
Infraestructura sin Compose	Dockerfiles propios, redes, volúmenes, secretos e inicializador	Construcción, arranque, reinicio y recreación de instancias
MongoDB con eventos inmutables	Proyección mutable de seguimiento, versionada y reconstruible	Actualización, duplicados, desorden, reconstrucción e índice
Adaptador de archivos sin flujo web completo	Selección, carga, verificación, estados y descarga privada	Prueba con bucket real, archivo inválido y URL expirada
Ejemplos estáticos de API	Captura optativa de solicitudes ejecutadas y verificación de persistencia	Inventario y expediente HTTP con fecha, versión y correlación

ID	Requisito comprometido	Implementación / evidencia
RF-01	Identidad, sesión y administración	ms_identidad_acceso; pruebas de seguridad y administración web
RF-02	Donantes, campañas, citas y procesamiento	ms_donantes; flujo de donación/procesamiento
RF-03	Receptores y solicitudes	ms_receptores; retenciones y cierre por recepción
RF-04	Inventario, disponibilidad y reservas	ms_inventario; concurrencia, idempotencia y recuperación
RF-05	Evaluación y dictamen humano	ms_compatibilidad; entradas y versiones conservadas
RF-06	Rutas, ranking y propuesta	ms_priorizacion_geografia; pruebas de matching y límites
RF-07	Traslado, ubicación y recepción	ms_transporte; flujo integrado y autorización por responsable
RF-08	Cadena de custodia	ms_custodia; orden, deduplicación y verificación de hashes
RF-09	Avisos y auditoría	ms_alertas, ms_auditoria; outbox, inbox y consulta autorizada
RF-10	Monitoreo y degradación	servicio_monitoreo; sondeos y pipelines documentales
RF-11	Archivos privados	Transporte, formulario web y GCS; pruebas de archivos reales
RNF-01	Autorización uniforme	RS256, Redis, permisos por rol/institución/recurso
RNF-02	Continuidad	Backend propio web, dos instancias por API,

ID	Requisito comprometido	Implementación / evidencia
		workers idempotentes
RNF-03	Contratos interoperables	OpenAPI, DTO JSON, XML/XSD, errores uniformes
RNF-04	Reproducibilidad	deployment/README.md, Compose y secretos generados
RNF-05	Recuperación y trazabilidad	Operaciones persistentes, correlación, auditoría y pruebas de fallos

La matriz de cierre determina qué criterios tienen evidencia final y cuáles conservan limitaciones. Las cifras históricas no se suman a las pruebas nuevas.

4. Arquitectura y continuidad

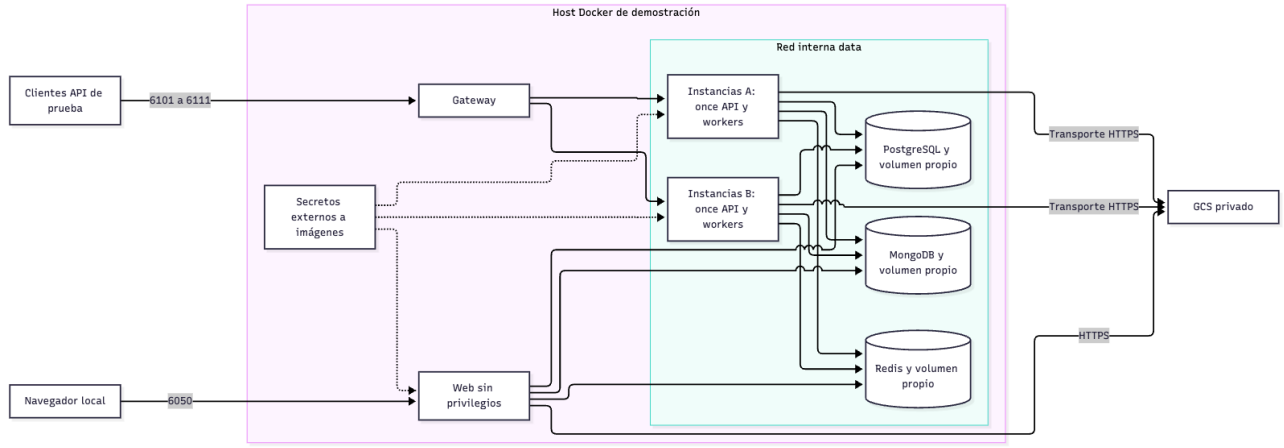


Componentes actuales

El navegador consume HTML y formularios del portal. El módulo de integración traduce cada operación a un contrato de aplicación. En modo monolith, ApplicationKernel ejecuta los módulos Flask mediante WSGI dentro del proceso, sin abrir listeners internos. El mismo controlador valida entradas, JWT, permisos Redis, reglas y transacciones. Las llamadas internas conservan firma de servicio y delegación del usuario. El modo HTTP se conserva como configuración explícita para pruebas y entornos anteriores; no es un fallback automático.

Cada módulo accede a PostgreSQL con su propia credencial limitada. El portal es una aplicación backend autorizada, no un cliente móvil con acceso directo a bases. Los microservicios no dependen de las vistas, plantillas ni sesiones privadas del portal. Las imágenes de dominio contienen únicamente su módulo y la biblioteca común; cada imagen puede actualizarse de forma individual.

La reutilización se fija mediante versión 0.3.0 y hashes de fuente en la evidencia. Una actualización de una API no modifica el código ya cargado en el portal. La compatibilidad de contratos y migraciones se debe conservar durante el despliegue gradual.



Despliegue y redes

Nginx descubre dos destinos por servicio mediante DNS Docker. Hay redundancia para Identidad y los otros diez propietarios. Cada instancia supervisa su API y su worker. La web ejecuta sus propios workers de todos los dominios. Los workers comparten colas SQL con bloqueo FOR UPDATE SKIP LOCKED; los consumidores deduplican eventos. La proyección MongoDB también admite duplicados y orden de entrega diferente.

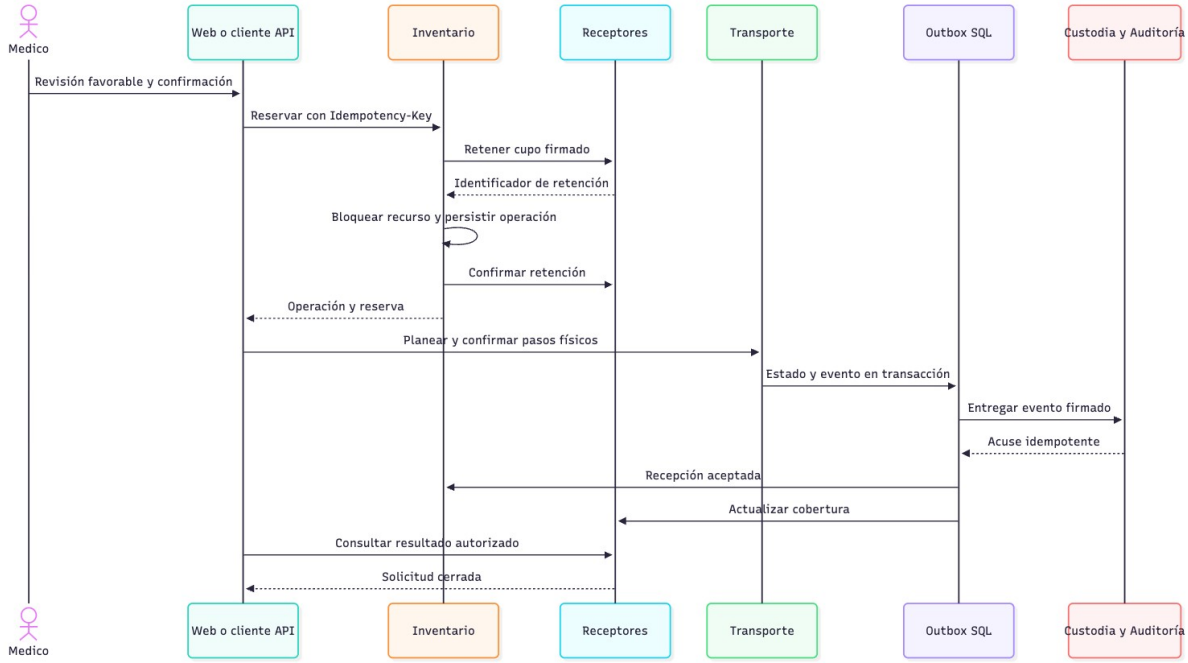
El gateway reintenta conexiones fallidas sin habilitar non_idempotent. Una respuesta perdida después de enviar una escritura tiene resultado incierto. El cliente conserva Idempotency-Key, consulta la operación y repite con la misma clave solo según su contrato. No cambia a otro backend después de escribir. Reservas y retenciones tienen identidad persistente y recuperación explícita.

Fallo	Portal modular	Clientes API	Recuperación
Una API o su worker	Opera con backend y workers propios	Instancia restante atiende el flujo	Reinicio supervisado y DNS
Todas las API de un servicio	Opera si sus dependencias de datos siguen disponibles	Fallan operaciones que requieren ese propietario	Restablecer instancia; consultar estado antes de repetir
Redis	Rechazo de operaciones protegidas	Igual	Restablecer Redis y verificar sesión; no omitir seguridad
PostgreSQL	Operaciones transaccionales indisponibles	Igual	Restablecer base y recuperar coordinaciones pendientes
MongoDB	Outbox SQL conserva eventos; consulta documental degradada	Igual	Reintento idempotente y reconstrucción de proyección
GCS	Funciones sin archivo disponibles; evidencia pendiente/error	Igual	Consultar estado y verificar objeto existente
Gateway	Portal modular continúa	API externa indisponible	Restaurar gateway; no hay réplica

Fallo	Portal modular	Clientes API	Recuperación
			de gateway en este alcance
Host Docker	Entorno indisponible	Entorno indisponible	Restaurar host, volúmenes y secretos

Dos contenedores en un host no equivalen a alta disponibilidad multizona. PostgreSQL, Redis, MongoDB, gateway y el host siguen siendo puntos únicos de infraestructura. Los resultados de disponibilidad se limitan al fallo de instancia comprobado.

5. Procesos integrados



Secuencia de reserva y traslado

Sangre y órganos

El origen registra un recurso y un médico lo libera con confirmación y motivo. El destino registra receptor y solicitud. Compatibilidad conserva la evaluación, candidatos, exclusiones, ruta y versiones de reglas. El médico dicta favorable o desfavorable para cada candidato. Priorización presenta un orden operativo explicable; la propuesta no reserva automáticamente.

Inventario crea una operación, solicita retención de capacidad a Receptores y bloquea el recurso. Las claves e índices impiden reservar dos veces el mismo recurso o cubrir en exceso una solicitud. Una confirmación pendiente se recupera; no se interpreta un timeout como ausencia de efecto.

Coordinación planea el traslado con ruta y responsable. Origen prepara, transportista recoge, transporta y entrega, y destino acepta. Cada paso exige estado y versión compatibles, perfil autorizado y confirmación humana. La recepción actualiza inventario y cobertura de solicitud mediante eventos durables. Custodia conserva secuencia y hashes; Alertas y Auditoría reciben el mismo contexto de correlación minimizado.

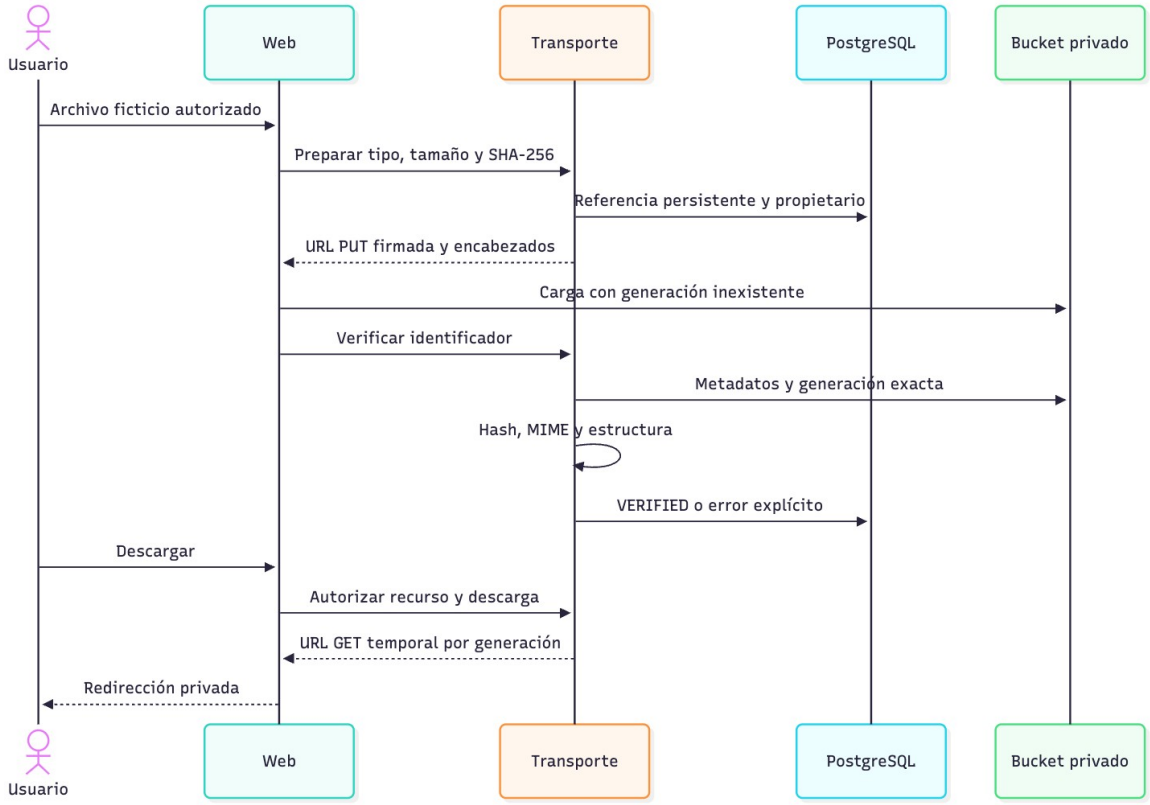
Para sangre RBC se conserva la regla documentada RBC-1.0. Para órganos, ORGAN-1.0 es una preselección académica por tipo y ABO idéntico, sin interpretación HLA. Los tres tipos de órgano existentes conservan pruebas separadas. No se infieren equivalencias entre sus ciclos clínicos.

Donación y procesamiento

Se registra el donante con consentimiento. Un médico documenta elegibilidad humana. La campaña define capacidad; la cita se registra transaccionalmente y no puede excederla. La donación conserva tipo, componente, fecha y confirmación. El

operador registra procesamiento. Un recurso de inventario puede vincular su origen a esa donación, conservando trazabilidad; la aceptación de un formulario no certifica aptitud clínica.

Archivos de un traslado



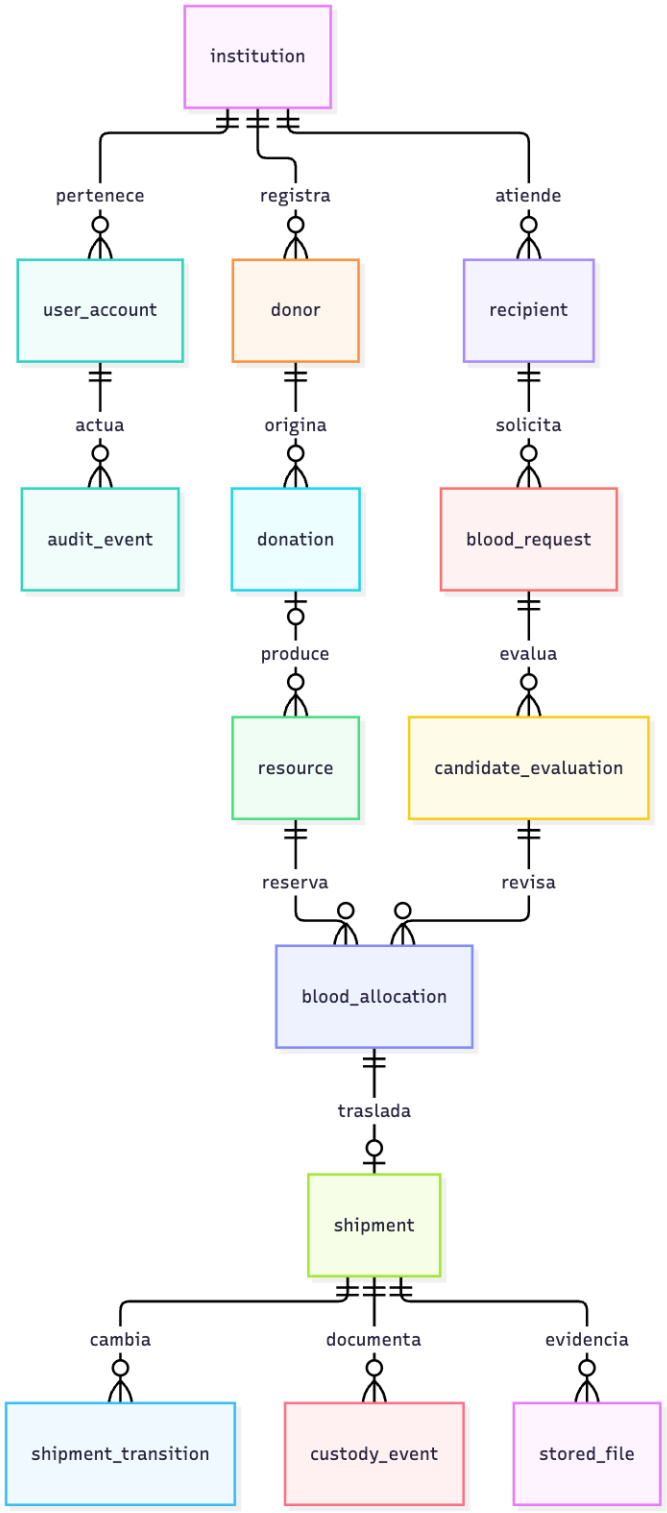
Carga privada de evidencia

El usuario autorizado selecciona PNG, JPEG o PDF de hasta 10 MiB. El portal calcula hash y prepara el registro en Transporte. El backend autoriza la asociación con el traslado, persiste la referencia y firma una carga de creación exclusiva. El encabezado firmado x-goog-if-generation-match: 0 evita sobrescribir un objeto existente.

Después de cargar, el servicio consulta metadatos y descarga la generación exacta para comprobar tamaño, SHA-256, tipo y estructura. Se limitan dimensiones de imágenes y páginas de PDF; se rechazan PDFs cifrados y mecanismos activos detectados. Esta validación estructural no sustituye un antivirus ni una política clínica de retención.

Los estados distinguen registro preparado, verificado, rechazado, ausente y error. Una descarga requiere autorización por recurso y estado verificado; se entrega una URL firmada temporal ligada a la generación. El bucket bloquea acceso público y no almacena datos reales de pacientes en estas pruebas.

6. Persistencia y modelo físico



Relaciones operativas

PostgreSQL

La base red_house conserva 71 tablas físicas en el esquema del dominio y una tabla técnica del portal en web_runtime, 72 en total. Deriva del esquema fijado de main y las ampliaciones descritas en Esquema compartido. Las vistas access_<servicio> adaptan los nombres del contrato sin duplicar registros. Cada credencial de ejecución dispone de

permisos únicamente sobre su ámbito. El propietario estructural no inicia sesión. Los usuarios de aplicación no reciben acceso administrativo a las tablas físicas.

El catálogo físico completo se genera desde pg_catalog e information_schema: columnas, tipos, obligatoriedad, valores predeterminados, claves, restricciones e índices. Incluye tablas conservadas como antecedente que las API no escriben; su existencia no se confunde con una función terminada. El archivo JSON de metadatos permite contrastar el catálogo con la base instalada.

Las tablas service_* incluyen propietario en su clave. Las vistas de outbox, inbox, decisiones, idempotencia, actividad y latido filtran ese propietario. Las funciones privilegiadas usan objetos calificados y search_path restringido. Los historiales mantienen prohibición de actualización/eliminación para las credenciales de ejecución.

El SQL de instalación se aplica explícitamente a una base vacía. Arrancar una API no destruye ni reconstruye tablas. Las migraciones de entornos anteriores, sus respaldos y la comparación histórica de filas permanecen documentados; el entorno habitual no se usa para fallos destructivos de esta entrega.

MongoDB

Transporte, Custodia, Auditoría y Monitoreo tienen bases documentales y credenciales propias. Los eventos se insertan por identificador estable. La entrega mediante outbox admite reintentos; una repetición con contenido distinto se rechaza. Los eventos de auditoría y custodia permanecen inmutables.

rh_transporte.shipment_tracking es una proyección mutable propiedad de Transporte. Su _id es el identificador del traslado PostgreSQL. Conserva schema_version, versión de secuencia, región, instituciones, transportista, reserva, último estado observado e identificador/fecha del evento. El evento de mayor secuencia reemplaza la proyección; uno anterior no la degrada. Una colisión de igual versión con distinto evento es conflicto. El campo last_state describe el último evento y no pretende sustituir la autoridad transaccional.

La migración documental crea índices regional_state y shipment_sequence. Las credenciales pueden actualizar la proyección, pero no los eventos. La reconstrucción recorre los eventos existentes sin editarlos. Las pruebas verifican inserción, consulta, actualización, filtro, pipeline, repetición, desorden, reconstrucción, permisos y un plan explain que utiliza el índice regional.

Monitoreo conserva sondeos con fuente, fecha, estado, tiempo, versión, dependencia y error. Sus pipelines calculan estado e historial. Primero intenta conservar cada sondeo en MongoDB y después en SQL; el endpoint puede consultar MongoDB si PostgreSQL falla, siempre que Redis permita autorizar al usuario. No promete conservación íntegra si ambos almacenes están caídos.

Redis

Redis mantiene sesiones activas, renovaciones, revocaciones, permisos versionados, contadores de consumo y datos temporales. Las ACL y prefijos delimitan acceso por servicio. JWT contiene identidad y ámbito, pero cada petición protegida comprueba el estado de sesión y autorización en Redis.

Los tokens de acceso duran 900 segundos; renovación, 86400 segundos en la configuración base. La renovación rota su secreto; reutilizar una renovación revoca la familia. Logout elimina credenciales del navegador y revoca la sesión. La web conserva tokens cifrados en Redis con TTL máximo de 86400 segundos y una ACL propia; la cookie contiene un identificador opaco. Las concesiones de renovación duran 60 segundos y se liberan solo por su propietario. PostgreSQL conserva cierres y revocaciones pendientes en web_runtime.session_revocations: una tabla técnica adicional a las 71 del dominio, 72 entre ambos esquemas. El cierre se registra antes de eliminar credenciales Redis; ante una caída de Redis, el worker completa la revocación después de recuperarse. Las marcas de cierre se conservan hasta 24 horas e impiden reutilizar una cookie antigua. Si PostgreSQL falla, la web deniega acceso protegido y logout elimina la cookie pero informa 503 al no poder confirmar el cierre global.

La caché de consultas autorizadas tiene TTL breve, incluye ámbito y revisión de datos. No se utiliza para decidir una reserva. Los bloqueos SQL y las restricciones únicas protegen el compromiso final. Rate limiting es compartido entre réplicas mediante Redis; detener una réplica no reinicia los contadores.

En la tabla siguiente, P representa el prefijo configurado mediante REDIS_PREFIX; los identificadores son variables del contrato, no credenciales publicadas.

Clave después de P	Contenido y finalidad	Expiración vigente
session:<sid>	Usuario y vencimiento de la familia activa	86400 s desde login; se elimina al revocar
refresh:<sha256>	Identificador de sesión asociado al secreto de renovación	Tiempo restante de la familia; rota sin extender su duración
used_refresh:<sha256>	Detecta reutilización y permite revocar la familia	Tiempo restante al rotar; la revocación explícita conserva marcador 86400 s
permissions:<user>	Perfil, ámbito, estado y versión autorizados	86400 s; se vuelve a publicar al autenticar/renovar
permission_version:<user>	Revisión vigente que invalida permisos anteriores	Sin TTL; cambia al modificar la cuenta
web:session:<sha256-cookie>	Credenciales cifradas del portal; ACL exclusiva	Hasta 86400 s
web:renewal:<sha256-cookie>	Concesión de renovación concurrente, dueño verificado	60 s
revoked:<jti>	Revocación del JWT presentado al cerrar sesión	Segundos restantes del token, mínimo 1 s
login_fail:<sha256>	Contador de accesos fallidos; bloquea desde cinco intentos	300 s desde el último intento contabilizado
limit:<service>:<category>:<scope>:<minute>	Contador atómico compartido de consumo	61 s
nonce:<receiver>:<caller>:<nonce>	Rechaza repetición de una firma interna	65 s; firma admite desfase máximo de 30 s
cache:<service>:read:<sha256>	Respuesta ligada a usuario, ámbito, filtros y revisión SQL	Inventario 10/15 s; catálogos 30 s

Las evidencias de login, renovación y revocación conservan comprobaciones SQL y de existencia/TTL Redis. Las pruebas adicionales verifican caché, límites, nonce repetido y denegación durante caída de Redis.

Google Cloud Storage

El proyecto autorizado es sistemas-integracion-01. El bucket red-house-evidencias-sistemas-integracion-01 usa Standard, us-central1, acceso uniforme y prevención de acceso público. La cuenta red-house-archivos recibió permisos de objetos limitados a ese bucket; la firma del usuario se limita a esa cuenta. Se utilizan credenciales impersonadas, sin descargar claves privadas de servicio.

PostgreSQL conserva identificador, traslado, propietario, ruta, MIME, tamaño, hash, generación y estado. Los archivos no se insertan en las bases. La prueba elimina únicamente sus objetos ficticios identificados; no vacía el bucket. Retención clínica, borrado de expedientes reales, antimalware y operación productiva requieren decisiones posteriores.

7. Servicios, contratos y comunicación

Las fichas individuales detallan propósito, dependencias, persistencia, endpoints, puertos, permisos, pruebas y artefactos de cada propietario. El expediente HTTP es navegable y distingue peticiones reales HTTP de invocaciones WSGI del monolito.

Todos los servicios usan Python/Flask, contratos /api/v1 y endpoints internos /internal/v1. Las respuestas externas ofrecen JSON o XML mediante Accept; el formato de entrada se determina con Content-Type. Los esquemas XSD validan estructura, cardinalidad y tipos, con entidades externas y DTD deshabilitados. El XML de respuesta utiliza el contrato tipado versionado; no se concatena XML con entradas sin escape.

El sobre satisfactorio contiene data y meta; el error contiene error.code, error.message, detalles permitidos y correlación. Las entradas se validan con modelos explícitos. Paginación y filtros rechazan campos o valores fuera de contrato. Los DTO de salida minimizan datos y excluyen credenciales. Los códigos distinguen validación, ausencia, autenticación, autorización, conflicto, límite y dependencia inaccesible.

Las llamadas internas exigen firma Ed25519 del servicio, marca temporal y nonce. Las llamadas con delegación conservan JWT del usuario. La autorización interna limita qué propietario puede invocar cada operación. Ni una ruta interna ni una dirección de red sustituyen esa autenticación.

Swagger se sirve en /docs/, con OpenAPI y ejemplos de entrada/salida, incluidos errores. Los ejemplos generados ayudan a construir solicitudes; únicamente el expediente de ejecución demuestra que se realizaron. Las capturas muestran resultados ejecutados y redactados, con acceso al JSON completo y persistencia asociada.

8. Seguridad y consistencia

Las contraseñas usan scrypt; no se conservan en texto plano. JWT acepta RS256, emisor y audiencia definidos, expiración y claims requeridos. Redis permite invalidar una sesión aunque el JWT todavía esté firmado y vigente. La autorización combina perfil, región, institución y relación con el recurso.

Los formularios web aplican CSRF, cookies de sesión y validación en servidor. Los archivos se validan antes de publicarse como verificados. SQL usa parámetros/ORM del controlador; las firmas internas y XML tienen defensas específicas contra repetición y XXE. Los errores no revelan DSN, SQL, claves o trazas al usuario.

Cada cambio conserva motivo, actor, correlación y evento local cuando corresponde. El evento se escribe junto con la transacción de negocio. Outbox desacopla su entrega a auditoría, alertas y proyecciones. Inbox deduplica por identificador y huella del contenido. Una caída posterior al commit no elimina el hecho persistido.

La evidencia HTTP es optativa y solo se habilita sobre datos ficticios. Redacta contraseñas, tokens, cookies, firmas y URLs privadas. Nunca se habilita por defecto en una instalación operativa. Los archivos .local, ADC y secretos quedan fuera de Git y del contexto Docker.

El Compose publica puertos solo en loopback para demostración local. No representa un despliegue público cifrado ni la certificación normativa de una plataforma clínica. Un despliegue externo requiere TLS, políticas de acceso/red, gestión de secretos y revisión regulatoria.

9. Algoritmos

La evaluación conserva reglas, versiones y exclusiones. El ranking operativo explica sus factores y no presenta ponderaciones médicas inventadas. La propuesta de balance utiliza matching bipartito Hopcroft–Karp: solicitudes/cupos en un lado, recursos candidatos en el otro; BFS identifica capas y DFS recorre caminos aumentantes. Su complejidad teórica es $O(E\sqrt{V})$ para el grafo representado.

El matching puede encontrar una cobertura mayor que una elección voraz cuando una solicitud admite varios recursos y otra solo uno. La salida es una propuesta; no modifica inventario ni reemplaza autorizaciones. Las pruebas comparan casos restrictivos, propuestas y ausencia de reservas automáticas. Problema, pseudocódigo, casos y límites adicionales están en Algoritmos.

Las mediciones anteriores de volumen y Locust se conservan como antecedentes identificados por fecha. No se atribuyen al código 0.3.0 sin repetición. La prueba de continuidad con carga moderada no equivale a carga sostenida, estrés o capacidad productiva certificada.

10. Instalación, operación y recuperación

El procedimiento ejecutable está en deployment/README.md. Requiere Docker Compose v2 y Python para preparar secretos. El instalador genera contraseñas, claves de firma y configuraciones privadas; crea la base compartida, ACL, usuarios documentales y 14 cuentas ficticias. Una repetición verifica versión y aplica únicamente la migración documental aditiva.

El ensayo desde una instantánea Git local clonada aprobó ocho etapas: preparación, construcción del instalador, arranque de almacenes, bootstrap, construcción de aplicaciones, arranque de 27 contenedores saludables, flujo de sangre con consultas JSON/XML de los once propietarios y repetición del bootstrap. Empleó secretos y volúmenes nuevos. El primer intento tuvo un fallo DNS del registro de imágenes; se conserva junto a la repetición aprobada. La evidencia del clon identifica la instantánea y los resultados.

El portal local Docker está en http://127.0.0.1:6050; gateway API en 6101–6111. PostgreSQL, Redis y MongoDB no publican puertos. La red data es interna. El acceso externo se limita a componentes que requieren GCS y al gateway/portal. Las aplicaciones se ejecutan sin privilegios, con sistema de archivos de solo lectura y áreas temporales explícitas.

Liveness comprueba el proceso. Readiness verifica dependencias aplicables y versión de esquema. El health check de cada contenedor verifica además el latido de su propio worker para que el latido de otra réplica no oculte su fallo. Los logs incluyen servicio, ruta, estado, tiempo y correlación, sin cuerpo ni credenciales.

El arranque se realiza por pares de réplicas, empezando por los consumidores de eventos, para permitir el drenaje de colas conservadas. El sondeo Docker usa un proceso Python mínimo, sin cargar paquetes de la aplicación. Las conexiones PostgreSQL se reutilizan en pools acotados por aplicación, credencial y esquema; un fallo no provoca la repetición automática de una escritura.

Una actualización compatible reemplaza primero A, espera salud y ejecuta una operación, después B. No se aplican cambios destructivos de esquema en el arranque. Cambiar contratos o persistencia requiere una migración planificada con compatibilidad temporal y respaldo. Recrear la misma versión prueba el mecanismo; no demuestra compatibilidad con una migración futura desconocida.

El ensayo final reemplazó ambas imágenes de Inventario por un ID distinto. Las 100 consultas continuas respondieron 200 y se verificó un alta SQL después de cada sustitución. La versión contractual permaneció en 0.3.0; esa prueba acredita la actualización compatible de este incremento.

El respaldo debe conservar PostgreSQL, documentos MongoDB, persistencia/ACL Redis, configuración privada, claves y sesiones web, además de los objetos GCS según su política. Restaurar solo tablas sin las claves o las colas puede dejar referencias y sesiones incoherentes. El procedimiento de respaldo y restauración aislada se detalla en la guía; los archivos con datos y secretos no forman parte del expediente público.

La restauración comprobada conserva 15 122 filas de 71 tablas, 7 632 documentos, cinco usuarios y cuatro roles MongoDB mediante conteos y hashes idénticos. Redis recuperó la clave de prueba con TTL de 3575 s. Se restauró en motores nuevos sin puertos publicados. Las generaciones GCS no se copian en ese respaldo local y deben conservarse en el bucket.

11. Pruebas, evidencia y criterios de aceptación

La suite usa PostgreSQL, MongoDB y Redis temporales, separados del entorno habitual. Los tests de contratos ejercitan JSON/XML/XSD, autenticación, autorización, expiración, renovación, revocación, errores, concurrencia, idempotencia, recuperación y permisos de almacenamiento. Los tests de continuidad detienen listeners HTTP y ejecutan sangre, órganos y procesamiento mediante el backend web autónomo.

Las pruebas Docker ejecutan operaciones de negocio a través del gateway; detienen las instancias A de los once propietarios, comprueban lectura JSON/XML y flujo de órganos, detienen todas las API para ejecutar flujos web, recrean réplicas secuencialmente y reinician las bases para comparar registros. Una página pública o un health check por sí solos no satisfacen esos criterios.

La prueba GCS usa el bucket autorizado, carga y descarga un PNG ficticio, comprueba metadatos SQL, rechazo de acceso público, autorización por traslado, objeto ausente, MIME/contenido inválido, prohibición de sobreescritura, firma expirada y degradación/recuperación. No utiliza URLs fabricadas.

Los resultados finales y sus tiempos se incorporan en Resultados de validación. Ese registro conserva esperado, obtenido y diagnóstico. Los límites de espera no se aumentan para ocultar un fallo. Una repetición posterior se identifica y no borra el resultado fallido previo.

La regresión final aprobó 182 pruebas, sin fallos ni omisiones, en 2005.90 s. Una prueba complementaria posterior aprobó en 36.08 s los errores de negociación y las conexiones fallidas/recuperadas de salud para los once propietarios. Docker aprobó nueve comprobaciones operativas. Chrome verificó 20 páginas de siete perfiles, las series Highcharts frente a sus datos del servidor, ancho 390 sin desbordamiento, una carga privada verificada y una consulta Swagger real.

El inventario final contiene 284 métodos/rutas con éxito HTTP y captura. Hay errores ejecutados para 240; las 44 rutas restantes sirven documentación fija y tienen una justificación explícita de no aplicabilidad de error de dominio. Todas las escrituras capturadas incluyen comprobación de persistencia. Estos números se obtienen del manifiesto de cobertura y no se fijan como contrato de futuras versiones.

La captura automática inventaría los métodos y rutas reales de cada aplicación. Cada registro incluye objetivo, URL, parámetros, headers redactados, petición, respuesta, código, fecha, versión, correlación, transporte y hash de

implementación. Las escrituras idempotentes se vinculan a una consulta del resultado comprometido en PostgreSQL. Las operaciones sin ese registro requieren la evidencia de flujo o de su almacén específico; el informe de cobertura señala las brechas en lugar de fabricar ejemplos.

12. Presentación y trabajo pendiente

El guion de demostración organiza responsabilidades y pasos reproducibles. No asigna nombres ni inventa participación. Incluye explicación del problema, arquitectura, flujo completo, JSON/XML, Swagger, Redis, MongoDB, archivos, Docker y fallos. El equipo completará reparto de participación, ensayo, grabación y publicación de YouTube.

La entrega académica completa requiere presentación presencial de todos los integrantes y un video accesible. Preparar el guion y los artefactos no acredita esos requisitos externos. Tampoco acredita validación clínica, operación productiva, pruebas de infraestructura multizona o certificación normativa.

13. Fuentes técnicas

- Reintentos y upstream de Nginx: distingue reintento de conexión y operaciones no idempotentes.
- Redes y DNS de Docker Compose: descubrimiento de servicios y recreación de contenedores.
- Roles IAM de Cloud Storage: alcance de permisos sobre objetos.
- Firmas de URLs de Cloud Storage: acceso temporal a objetos privados.
- Enunciado y matriz de ejecución, reglas del proyecto y modelo compartido: fuentes internas de alcance y trazabilidad.

Referencias y archivos de esta sección

Algoritmos: documentation/markdowns/Algoritmos.md

Enunciado y matriz de ejecución: documentation/markdowns/Cierre_segundo_parcial.md

Esquema compartido: documentation/markdowns/Esquema_compartido.md

Firmas de URLs de Cloud Storage: https://docs.cloud.google.com/storage/docs/access-control/signing-urls-with-helpers

Redes y DNS de Docker Compose: https://docs.docker.com/compose/how-tos/networking/

Reintentos y upstream de Nginx: https://nginx.org/en/docs/http/ngx_http_proxy_module.html#proxy_next_upstream

Resultados de validación: documentation/evidencias/segundo-parcial/RESULTADOS.md

Roles IAM de Cloud Storage: https://docs.cloud.google.com/storage/docs/access-control/iam-roles

catálogo físico completo: documentation/segundo-parcial/Modelo_fisico.md

deployment/README.md: deployment/README.md

evidencia del clon: documentation/evidencias/segundo-parcial/clean-clone.json

expediente HTTP: documentation/evidencias/segundo-parcial/http/index.html

fichas individuales: documentation/segundo-parcial/Fichas_microservicios.md

guion de demostración: stuff/presentacion/Guion_segundo_parcial.md

matriz de cierre: documentation/markdowns/Cierre_segundo_parcial.md

modelo compartido: documentation/markdowns/Esquema_compartido.md

reglas del proyecto: documentation/markdowns/Reglas_de_negocio.md

validación de sesiones: documentation/evidencias/sesiones-redis/RESULTADOS.md

Fichas individuales de microservicios

Edición 0.3.0. Los contratos enlazados contienen campos, tipos, obligatoriedad, ejemplos JSON/XML, XSD, errores y paginación. El expediente HTTP distingue ejemplos de ejecuciones.

Cada servicio tiene API Flask/REST y worker supervisado, módulos de rutas/modelos, biblioteca común de autorización/contratos y persistencia propia mediante vistas limitadas. Puerto interno 8080; el gateway publica un puerto

por propietario. Versiones exactas de dependencias en requirements-verified.txt; Docker usa Python 3.13 y las pruebas locales Python 3.12.

Identidad y acceso

Identifica a cada persona y limita qué puede hacer y en qué institución.

Justificación y proceso: Un médico inicia sesión, recibe JWT y renovación; Inventario verifica su sesión y permisos antes de reservar.

Integración declarada: puede invocar contratos internos de auditoria; acepta llamadas firmadas de donantes, inventario, priorizacion_geografia, transporte. Estas relaciones incluyen entrega de eventos. Monitoreo consulta además la salud pública de todos los componentes. Cada ruta conserva su lista específica de consumidores y delegación en OpenAPI.

Persistencia y relaciones: Usuarios, instituciones, sesiones registradas y hashes scrypt en PostgreSQL. Redis conserva sesiones activas, permisos versionados, renovaciones y revocaciones.

Ámbito SQL: access_identidad_acceso. La correspondencia siguiente procede del SQL de instalación. Las tablas técnicas service_* filtran el propietario; las claves y relaciones completas constan en el catálogo físico.

Vista autorizada	Tabla física compartida
activity_events	red_house.service_activity_events
cache_revision	red_house.cache_revision
catalogue	red_house.catalogue_entry
decisions	red_house.service_decisions
idempotency	red_house.service_idempotency
inbox	red_house.service_inbox
institutions	red_house.institution
outbox	red_house.service_outbox
schema_version	red_house.service_schema_version
sessions	red_house.web_session
users	red_house.user_account
worker_health	red_house.service_worker_health

Despliegue: Dockerfile, dos instancias identidad-acceso-a/b, gateway http://127.0.0.1:6101 y Swagger /docs/. Configuración privada propia: DATABASE_URL, REDIS_URL, claves públicas JWT y de servicios, clave privada de firma interna y URLs del gateway. Solo Identidad conserva la clave privada JWT. Transporte añade configuración GCS; los cuatro propietarios documentales añaden MongoDB.

Seguridad: JWT RS256 con Redis en cada operación protegida; rol y recurso según contrato. Rutas internas exigen firma, nonce y autoridad llamante. POST/PATCH de negocio conservan Idempotency-Key donde está declarado, versión y motivo cuando corresponde. GET autorizado genera auditoría; las transacciones publican eventos por outbox.

Contratos: OpenAPI completo · Directorio JSON/XML/XSD · Ejecuciones HTTP.

Método	Ruta	Objetivo	Códigos declarados
POST	/api/v1/auth/login	Iniciar sesión independiente	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
POST	/api/v1/auth/refresh	Rotar renovación; repetición revoca familia	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
POST	/api/v1/auth/logout	Cerrar sesión y revocar familia	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
POST	/api/v1/auth/revoke	Revocar familia con renovación	200, 400, 401, 403, 404, 406, 409,

Método	Ruta	Objetivo	Códigos declarados
		opaca, incluso si venció el JWT de acceso	413, 415, 422, 429, 500, 503
GET	/api/v1/auth/me	Consultar perfil y permisos vigentes	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/api/v1/institutions	Instituciones de la región; sin datos clínicos	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
POST	/api/v1/institutions	Registrar institución de la región	201, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/api/v1/institutions/{identifier}	Detalle administrativo de institución regional	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
PATCH	/api/v1/institutions/{identifier}	Editar nombre institucional propio con versión y motivo	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
POST	/api/v1/institutions/{identifier}/administrator	Provisionar primer administrador de una institución creada por esta cuenta	201, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
POST	/api/v1/users	Crear cuenta institucional	201, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/api/v1/users	Cuentas propias o personal regional de traslado; nunca hashes ni sesiones	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
PATCH	/api/v1/users/{identifier}	Activar/desactivar cuenta; invalidar permisos antes del cambio	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/internal/v1/institutions/{identifier}	Referencia institucional autorizada	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/internal/v1/users/{identifier}	Validar personal de traslado	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/api/v1/catalogues/sites	Consultar sites	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
POST	/api/v1/catalogues/sites	Registrar sites	201, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/api/v1/catalogues/sites/{identifier}	Detalle de sites	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
PATCH	/api/v1/catalogues/sites/{identifier}	Editar o desactivar sites	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/api/v1/catalogues/contacts	Consultar contacts	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
POST	/api/v1/catalogues/contacts	Registrar contacts	201, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/api/v1/catalogues/contacts/{identifier}	Detalle de contacts	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
PATCH	/api/v1/catalogues/contacts/{identifier}	Editar o desactivar contacts	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/api/v1/catalogues/schedules	Consultar schedules	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
POST	/api/v1/catalogues/schedules	Registrar schedules	201, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/api/v1/catalogues/schedules/{identifier}	Detalle de schedules	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
PATCH	/api/v1/catalogues/schedules/	Editar o desactivar schedules	200, 400, 401, 403, 404, 406, 409,

Método	Ruta	Objetivo	Códigos declarados
	{identifier}		413, 415, 422, 429, 500, 503
GET	/api/v1/catalogues/capacities	Consultar capacities	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
POST	/api/v1/catalogues/capacities	Registrar capacities	201, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/api/v1/catalogues/capacities/{identifier}	Detalle de capacities	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
PATCH	/api/v1/catalogues/capacities/{identifier}	Editar o desactivar capacities	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/api/v1/catalogues/locations	Consultar locations	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
POST	/api/v1/catalogues/locations	Registrar locations	201, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/api/v1/catalogues/locations/{identifier}	Detalle de locations	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
PATCH	/api/v1/catalogues/locations/{identifier}	Editar o desactivar locations	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/internal/v1/locations/{identifier}	Validar ubicación y sede de la institución del usuario	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503

Entradas y salidas: encabezados Authorization, Accept, Content-Type, X-Correlation-ID y Idempotency-Key según operación. Parámetros de ruta UUID y consulta/paginación constan en OpenAPI. Respuesta {data,meta} o {error,meta}, con representación XML equivalente. El contrato interno añade encabezados de firma del servicio; los valores secretos están redactados en evidencia.

Pruebas: suite de apps/services/tests, expediente dinámico por método/ruta y pruebas Docker de negocio. Consultar RESULTADOS.md para resultados reales y limitaciones; no se considera probado un endpoint solo por aparecer en esta tabla.

Donantes

Gestiona registros preliminares, consentimiento, dictamen humano, donaciones, campañas y citas.

Justificación y proceso: Un donante se registra y agenda una cita. Un médico registra elegibilidad; después se documentan recolección y procesamiento.

Integración declarada: puede invocar contratos internos de alertas, auditoria, identidad_acceso; acepta llamadas firmadas de inventario. Estas relaciones incluyen entrega de eventos. Monitoreo consulta además la salud pública de todos los componentes. Cada ruta conserva su lista específica de consumidores y delegación en OpenAPI.

Persistencia y relaciones: PostgreSQL conserva expedientes y transacciones. Redis valida acceso y limita consumo. Outbox publica auditoría y avisos dirigidos al donante.

Ámbito SQL: access_donantes. La correspondencia siguiente procede del SQL de instalación. Las tablas técnicas service_* filtran el propietario; las claves y relaciones completas constan en el catálogo físico.

Vista autorizada	Tabla física compartida
activity_events	red_house.service_activity_events
appointments	red_house.appointment
campaigns	red_house.campaign
decisions	red_house.service_decisions
donation_steps	red_house.donation_event

Vista autorizada	Tabla física compartida
donations	red_house.donation
donor_reviews	red_house.donor_review
donors	red_house.donor
idempotency	red_house.service_idempotency
inbox	red_house.service_inbox
outbox	red_house.service_outbox
schema_version	red_house.service_schema_version
worker_health	red_house.service_worker_health

Despliegue: Dockerfile, dos instancias donantes-a/b, gateway http://127.0.0.1:6102 y Swagger /docs/. Configuración privada propia: DATABASE_URL, REDIS_URL, claves públicas JWT y de servicios, clave privada de firma interna y URLs del gateway. Solo Identidad conserva la clave privada JWT. Transporte añade configuración GCS; los cuatro propietarios documentales añaden MongoDB.

Seguridad: JWT RS256 con Redis en cada operación protegida; rol y recurso según contrato. Rutas internas exigen firma, nonce y autoridad llamante. POST/PATCH de negocio conservan Idempotency-Key donde está declarado, versión y motivo cuando corresponde. GET autorizado genera auditoría; las transacciones publican eventos por outbox.

Contratos: OpenAPI completo · Directorio JSON/XML/XSD · Ejecuciones HTTP.

Método	Ruta	Objetivo	Códigos declarados
POST	/api/v1/donors	Registro preliminar con consentimiento	201, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/api/v1/donors	Expedientes autorizados	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/api/v1/donors/{identifier}	Expediente y evaluaciones humanas	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
PATCH	/api/v1/donors/{identifier}	Corregir expediente sin dictámenes ni donaciones	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
POST	/api/v1/donors/{identifier}/reviews	Dictamen humano de elegibilidad	201, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
POST	/api/v1/campaigns	Crear campaña y capacidad de citas	201, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/api/v1/campaigns	Campañas institucionales vigentes	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
POST	/api/v1/appointments	Reservar cita sin exceder capacidad	201, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/api/v1/appointments	Citas propias o institucionales	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
POST	/api/v1/appointments/{identifier}/cancel	Cancelar cita y liberar cupo	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
POST	/api/v1/donations	Registrar donación tras elegibilidad y confirmación humana	201, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/api/v1/donations	Donaciones de la institución sin expediente personal	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
POST	/api/v1/donations/{identifier}/steps	Documentar procesamiento o rechazo; no infiere elegibilidad clínica	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/internal/v1/donations/{identifier}	Referencia de procedencia; sin	200, 400, 401, 403, 404, 406, 409,

Método	Ruta	Objetivo	Códigos declarados
		identidad ni estudios del donante	413, 415, 422, 429, 500, 503
GET	/api/v1/donations/{identifier}	Donación y pasos de procesamiento institucionales	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
PATCH	/api/v1/donations/{identifier}	Corregir fecha antes de procesamiento, conservando motivo	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/api/v1/summary	Indicadores agregados autorizados; no expone expedientes	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/api/v1/campaigns/{identifier}	Detalle de campaña institucional	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
PATCH	/api/v1/campaigns/{identifier}	Editar campaña sin reducir cupos ya ocupados	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503

Entradas y salidas: encabezados Authorization, Accept, Content-Type, X-Correlation-ID y Idempotency-Key según operación. Parámetros de ruta UUID y consulta/paginación constan en OpenAPI. Respuesta {data,meta} o {error,meta}, con representación XML equivalente. El contrato interno añade encabezados de firma del servicio; los valores secretos están redactados en evidencia.

Pruebas: suite de apps/services/tests, expediente dinámico por método/ruta y pruebas Docker de negocio. Consultar RESULTADOS.md para resultados reales y limitaciones; no se considera probado un endpoint solo por aparecer en esta tabla.

Receptores y solicitudes

Administra expedientes de receptores y el ciclo de vida y cobertura de solicitudes.

Justificación y proceso: Una solicitud de una unidad retiene un cupo antes de que Inventario reserve el recurso. Dos peticiones concurrentes no pueden cubrir dos veces ese cupo.

Integración declarada: puede invocar contratos internos de alertas, auditoria; acepta llamadas firmadas de compatibilidad, inventario. Estas relaciones incluyen entrega de eventos. Monitoreo consulta además la salud pública de todos los componentes. Cada ruta conserva su lista específica de consumidores y delegación en OpenAPI.

Persistencia y relaciones: PostgreSQL conserva receptores, solicitudes y retenciones de capacidad; Redis valida sesión y permisos. Auditoría/alertas reciben eventos durables.

Ámbito SQL: access_receptores. La correspondencia siguiente procede del SQL de instalación. Las tablas técnicas service_* filtran el propietario; las claves y relaciones completas constan en el catálogo físico.

Vista autorizada	Tabla física compartida
activity_events	red_house.service_activity_events
decisions	red_house.service_decisions
holds	red_house.request_hold
idempotency	red_house.service_idempotency
inbox	red_house.service_inbox
outbox	red_house.service_outbox
recipients	red_house.recipient
requests	red_house.blood_request
schema_version	red_house.service_schema_version
worker_health	red_house.service_worker_health

Despliegue: Dockerfile, dos instancias receptores-a/b, gateway http://127.0.0.1:6103 y Swagger /docs/. Configuración privada propia: DATABASE_URL, REDIS_URL, claves públicas JWT y de servicios, clave privada de firma interna y URLs del gateway. Solo Identidad conserva la clave privada JWT. Transporte añade configuración GCS; los cuatro propietarios documentales añaden MongoDB.

Seguridad: JWT RS256 con Redis en cada operación protegida; rol y recurso según contrato. Rutas internas exigen firma, nonce y autoridad llamante. POST/PATCH de negocio conservan Idempotency-Key donde está declarado, versión y motivo cuando corresponde. GET autorizado genera auditoría; las transacciones publican eventos por outbox.

Contratos: OpenAPI completo · Directorio JSON/XML/XSD · Ejecuciones HTTP.

Método	Ruta	Objetivo	Códigos declarados
POST	/api/v1/recipients	Expediente diferenciado de sangre u órgano	201, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/api/v1/recipients	Expedientes de la propia institución	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/api/v1/recipients/{identifier}	Consultar expediente clínico autorizado	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
PATCH	/api/v1/recipients/{identifier}	Corregir receptor antes de registrar solicitudes	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
POST	/api/v1/requests	Solicitud humana de recursos	201, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/api/v1/requests	Solicitudes institucionales y cobertura	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/api/v1/requests/{identifier}	Solicitud y reservas pendientes/confirmadas	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
PATCH	/api/v1/requests/{identifier}	Editar solicitud abierta sin retenciones, invalidando evaluaciones anteriores	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
POST	/api/v1/requests/{identifier}/cancel	Cancelar únicamente sin reservas pendientes o físicas	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/internal/v1/requests/{identifier}	Instantánea mínima para compatibilidad y reserva	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
POST	/internal/v1/holds	Retener capacidad; serializar solicitud y receptor	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
POST	/internal/v1/holds/{identifier}	Confirmar, liberar o recibir retención de forma idempotente	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/internal/v1/holds-by-operation/{identifier}	Localizar retención tras perder una respuesta	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
POST	/internal/v1/events	Recibir recepción o liberación sin duplicados	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/api/v1/summary	Indicadores agregados autorizados; no expone expedientes	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503

Entradas y salidas: encabezados Authorization, Accept, Content-Type, X-Correlation-ID y Idempotency-Key según operación. Parámetros de ruta UUID y consulta/paginación constan en OpenAPI. Respuesta {data,meta} o {error,meta}, con representación XML equivalente. El contrato interno añade encabezados de firma del servicio; los valores secretos están redactados en evidencia.

Pruebas: suite de apps/services/tests, expediente dinámico por método/ruta y pruebas Docker de negocio. Consultar RESULTADOS.md para resultados reales y limitaciones; no se considera probado un endpoint solo por aparecer en esta tabla.

Inventario y reservas

Mantiene unidades de sangre y ofertas de órganos, su disponibilidad, reservas y ubicación institucional.

Justificación y proceso: El origen registra y libera un recurso; el destino lo reserva con evaluación y dictamen favorable; tras recepción queda registrado en la institución de destino.

Integración declarada: puede invocar contratos internos de alertas, auditoria, compatibilidad, donantes, identidad_acceso, priorizacion_geografia, receptores; acepta llamadas firmadas de compatibilidad, transporte. Estas relaciones incluyen entrega de eventos. Monitoreo consulta además la salud pública de todos los componentes. Cada ruta conserva su lista específica de consumidores y delegación en OpenAPI.

Persistencia y relaciones: PostgreSQL es autoridad de recursos, movimientos y operaciones. Redis ofrece caché breve de disponibilidad y seguridad; una reserva siempre reconsulta PostgreSQL.

Ámbito SQL: access_inventario. La correspondencia siguiente procede del SQL de instalación. Las tablas técnicas service_* filtran el propietario; las claves y relaciones completas constan en el catálogo físico.

Vista autorizada	Tabla física compartida
activity_events	red_house.service_activity_events
cache_revision	red_house.cache_revision
decisions	red_house.service_decisions
idempotency	red_house.service_idempotency
inbox	red_house.service_inbox
movements	red_house.blood_movement
operations	red_house.allocation_operation
outbox	red_house.service_outbox
reservations	red_house.blood_allocation
resource_reviews	red_house.resource_review
resources	red_house.resource
schema_version	red_house.service_schema_version
worker_health	red_house.service_worker_health

Despliegue: Dockerfile, dos instancias inventario-a/b, gateway http://127.0.0.1:6104 y Swagger /docs/. Configuración privada propia: DATABASE_URL, REDIS_URL, claves públicas JWT y de servicios, clave privada de firma interna y URLs del gateway. Solo Identidad conserva la clave privada JWT. Transporte añade configuración GCS; los cuatro propietarios documentales añaden MongoDB.

Seguridad: JWT RS256 con Redis en cada operación protegida; rol y recurso según contrato. Rutas internas exigen firma, nonce y autoridad llamante. POST/PATCH de negocio conservan Idempotency-Key donde está declarado, versión y motivo cuando corresponde. GET autorizado genera auditoría; las transacciones publican eventos por outbox.

Contratos: OpenAPI completo · Directorio JSON/XML/XSD · Ejecuciones HTTP.

Método	Ruta	Objetivo	Códigos declarados
POST	/api/v1/resources	Registrar unidad u oferta separada, inicialmente en cuarentena	201, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/api/v1/resources	Inventario propio; coordinador recibe disponibilidad regional mínima	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
POST	/api/v1/resources/{identifier}/release	Liberación humana documentada	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503

Método	Ruta	Objetivo	Códigos declarados
GET	/api/v1/availability	Disponibilidad regional minimizada con caché autorizada	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/internal/v1/available-resources	Fuente autoritativa de candidatos; omite caché	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
POST	/api/v1/reservations	Reservar con retención de capacidad y seguimiento de operación	202, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/api/v1/reservations	Reservas institucionales o regionales para coordinación	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/api/v1/operations/{identifier}	Consultar resultado tras pérdida de respuesta	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
POST	/api/v1/operations/{identifier}/resume	Reanudar confirmación o compensación pendiente	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/api/v1/reservations/{identifier}	Consultar asignación autorizada	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
POST	/api/v1/reservations/{identifier}/cancel	Liberar reserva antes de programar un traslado	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/internal/v1/reservations/{identifier}	Datos mínimos para planear traslado	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
POST	/internal/v1/reservations/{identifier}/claim	Vincular traslado durable antes de permitir recolección	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
POST	/internal/v1/events	Aplicar hechos logísticos en orden y sin duplicación	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/api/v1/resources/{identifier}	Recurso y movimientos; sin identidad del donante	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
PATCH	/api/v1/resources/{identifier}	Corregir unidad antes de liberación clínica	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/api/v1/summary	Indicadores agregados autorizados; no expone expedientes	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
POST	/api/v1/resources/{identifier}/withdraw	Baja lógica sin reservas ni traslado; conserva historial	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503

Entradas y salidas: encabezados Authorization, Accept, Content-Type, X-Correlation-ID y Idempotency-Key según operación. Parámetros de ruta UUID y consulta/paginación constan en OpenAPI. Respuesta {data,meta} o {error,meta}, con representación XML equivalente. El contrato interno añade encabezados de firma del servicio; los valores secretos están redactados en evidencia.

Pruebas: suite de apps/services/tests, expediente dinámico por método/ruta y pruebas Docker de negocio. Consultar RESULTADOS.md para resultados reales y limitaciones; no se considera probado un endpoint solo por aparecer en esta tabla.

Compatibilidad

Genera candidatos explicables y conserva evaluaciones y dictámenes humanos.

Justificación y proceso: Para RBC se aplica el filtro ABO/RhD ya documentado. Para órgano se preselecciona mismo tipo y ABO idéntico, sin interpretar HLA.

Integración declarada: puede invocar contratos internos de auditoria, inventario, priorizacion_geografia, receptores; acepta llamadas firmadas de inventario, priorizacion_geografia. Estas relaciones incluyen entrega de eventos. Monitoreo consulta además la salud pública de todos los componentes. Cada ruta conserva su lista específica de consumidores y delegación en OpenAPI.

Persistencia y relaciones: PostgreSQL conserva instantáneas, versiones, candidatos y revisión. Flask consulta propietarios por HTTP; Redis valida JWT y permisos.

Ámbito SQL: access_compatibilidad. La correspondencia siguiente procede del SQL de instalación. Las tablas técnicas service_* filtran el propietario; las claves y relaciones completas constan en el catálogo físico.

Vista autorizada	Tabla física compartida
activity_events	red_house.service_activity_events
decisions	red_house.service_decisions
evaluations	red_house.candidate_evaluation
idempotency	red_house.service_idempotency
inbox	red_house.service_inbox
outbox	red_house.service_outbox
reviews	red_house.blood_review
schema_version	red_house.service_schema_version
worker_health	red_house.service_worker_health

Despliegue: Dockerfile, dos instancias compatibilidad-a/b, gateway http://127.0.0.1:6105 y Swagger /docs/. Configuración privada propia: DATABASE_URL, REDIS_URL, claves públicas JWT y de servicios, clave privada de firma interna y URLs del gateway. Solo Identidad conserva la clave privada JWT. Transporte añade configuración GCS; los cuatro propietarios documentales añaden MongoDB.

Seguridad: JWT RS256 con Redis en cada operación protegida; rol y recurso según contrato. Rutas internas exigen firma, nonce y autoridad llamante. POST/PATCH de negocio conservan Idempotency-Key donde está declarado, versión y motivo cuando corresponde. GET autorizado genera auditoría; las transacciones publican eventos por outbox.

Contratos: OpenAPI completo · Directorio JSON/XML/XSD · Ejecuciones HTTP.

Método	Ruta	Objetivo	Códigos declarados
POST	/api/v1/evaluations	Preselección ilustrativa con datos y reglas versionados	201, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/api/v1/evaluations	Historial institucional de evaluaciones y versiones	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/api/v1/evaluations/{identifier}	Consultar evaluación, explicación y dictámenes	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
POST	/api/v1/evaluations/{identifier}/reviews	Registrar dictamen humano inmutable de un candidato	201, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/internal/v1/evaluations/{identifier}	Instantánea y dictámenes para reservar o priorizar	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503

Entradas y salidas: encabezados Authorization, Accept, Content-Type, X-Correlation-ID y Idempotency-Key según operación. Parámetros de ruta UUID y consulta/paginación constan en OpenAPI. Respuesta {data,meta} o {error,meta}, con representación XML equivalente. El contrato interno añade encabezados de firma del servicio; los valores secretos están redactados en evidencia.

Pruebas: suite de apps/services/tests, expediente dinámico por método/ruta y pruebas Docker de negocio. Consultar RESULTADOS.md para resultados reales y limitaciones; no se considera probado un endpoint solo por aparecer en esta tabla.

Priorización y geografía

Mantiene estimaciones de rutas, ranking operativo y propuestas de cobertura máxima.

Justificación y proceso: Si un paciente admite A o B y otro solo A, un recorrido voraz puede dejar uno sin propuesta. Un camino aumentante permite proponer B al flexible y A al restringido.

Integración declarada: puede invocar contratos internos de auditoria, compatibilidad, identidad_acceso; acepta llamadas firmadas de compatibilidad, inventario, transporte. Estas relaciones incluyen entrega de eventos. Monitoreo consulta además la salud pública de todos los componentes. Cada ruta conserva su lista específica de consumidores y delegación en OpenAPI.

Persistencia y relaciones: PostgreSQL conserva rutas versionadas, factores, rankings y propuestas. Python ejecuta ordenamiento y matching bipartito Hopcroft–Karp. Redis aplica seguridad.

Ámbito SQL: access_priorizacion_geografia. La correspondencia siguiente procede del SQL de instalación. Las tablas técnicas service_* filtran el propietario; las claves y relaciones completas constan en el catálogo físico.

Vista autorizada	Tabla física compartida
activity_events	red_house.service_activity_events
decisions	red_house.service_decisions
idempotency	red_house.service_idempotency
inbox	red_house.service_inbox
outbox	red_house.service_outbox
proposals	red_house.balance_proposal
rankings	red_house.ranking_run
routes	red_house.regional_route
schema_version	red_house.service_schema_version
worker_health	red_house.service_worker_health

Despliegue: Dockerfile, dos instancias priorizacion-geografia-a/b, gateway http://127.0.0.1:6106 y Swagger /docs/. Configuración privada propia: DATABASE_URL, REDIS_URL, claves públicas JWT y de servicios, clave privada de firma interna y URLs del gateway. Solo Identidad conserva la clave privada JWT. Transporte añade configuración GCS; los cuatro propietarios documentales añaden MongoDB.

Seguridad: JWT RS256 con Redis en cada operación protegida; rol y recurso según contrato. Rutas internas exigen firma, nonce y autoridad llamante. POST/PATCH de negocio conservan Idempotency-Key donde está declarado, versión y motivo cuando corresponde. GET autorizado genera auditoría; las transacciones publican eventos por outbox.

Contratos: OpenAPI completo · Directorio JSON/XML/XSD · Ejecuciones HTTP.

Método	Ruta	Objetivo	Códigos declarados
POST	/api/v1/routes	Registrar estimación logística con procedencia	201, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/api/v1/routes	Rutas autorizadas de la región	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
PATCH	/api/v1/routes/{identifier}	Actualizar ruta con control de versión	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/api/v1/routes/{identifier}	Consultar una ruta regional por identificador	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/internal/v1/routes/{identifier}	Revalidar una ruta conocida	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/internal/v1/route-options	Estimaciones regionales para filtrar candidatos	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
POST	/api/v1/rankings	Ranking operativo explicable y	201, 400, 401, 403, 404, 406, 409,

Método	Ruta	Objetivo	Códigos declarados
		versionado, no decisión clínica	413, 415, 422, 429, 500, 503
GET	/api/v1/rankings	Historial propio de rankings operativos	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/api/v1/rankings/{identifier}	Factores, explicación y resultados del ranking	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
POST	/api/v1/allocation-proposals	Matching máximo de candidatos con dictamen favorable; propuesta sin reservas	201, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/api/v1/allocation-proposals	Historial institucional de propuestas de cobertura, sin reservas automáticas	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/api/v1/allocation-proposals/{identifier}	Consultar propuesta, límites y versiones originales	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503

Entradas y salidas: encabezados Authorization, Accept, Content-Type, X-Correlation-ID y Idempotency-Key según operación. Parámetros de ruta UUID y consulta/paginación constan en OpenAPI. Respuesta {data,meta} o {error,meta}, con representación XML equivalente. El contrato interno añade encabezados de firma del servicio; los valores secretos están redactados en evidencia.

Pruebas: suite de apps/services/tests, expediente dinámico por método/ruta y pruebas Docker de negocio. Consultar RESULTADOS.md para resultados reales y limitaciones; no se considera probado un endpoint solo por aparecer en esta tabla.

Transporte

Planea y sigue traslados, responsables, incidencias, ubicación autorizada y evidencias.

Justificación y proceso: Coordinación planea; origen prepara; traslado recoge y entrega; destino acepta. Cada paso genera eventos de custodia y auditoría.

Integración declarada: puede invocar contratos internos de alertas, auditoria, custodia, identidad_acceso, inventario, priorizacion_geografia; acepta llamadas firmadas de custodia. Estas relaciones incluyen entrega de eventos. Monitoreo consulta además la salud pública de todos los componentes. Cada ruta conserva su lista específica de consumidores y delegación en OpenAPI.

Persistencia y relaciones: PostgreSQL conserva órdenes, pasos y referencias de archivos. MongoDB recibe documentos y telemetría mediante outbox. Redis conserva sesión y referencia de ubicación reciente. GCS almacena archivos privados; el portal carga, verifica y descarga mediante firmas temporales.

Ámbito SQL: access_transporte. La correspondencia siguiente procede del SQL de instalación. Las tablas técnicas service_* filtran el propietario; las claves y relaciones completas constan en el catálogo físico.

Vista autorizada	Tabla física compartida
activity_events	red_house.service_activity_events
decisions	red_house.service_decisions
files	red_house.stored_file
idempotency	red_house.service_idempotency
inbox	red_house.service_inbox
outbox	red_house.service_outbox
schema_version	red_house.service_schema_version
shipments	red_house.shipment
telemetry	red_house.transport_telemetry

Vista autorizada	Tabla física compartida
transitions	red_house.shipment_transition
worker_health	red_house.service_worker_health

Despliegue: Dockerfile, dos instancias transporte-a/b, gateway http://127.0.0.1:6107 y Swagger /docs/. Configuración privada propia: DATABASE_URL, REDIS_URL, claves públicas JWT y de servicios, clave privada de firma interna y URLs del gateway. Solo Identidad conserva la clave privada JWT. Transporte añade configuración GCS; los cuatro propietarios documentales añaden MongoDB.

Seguridad: JWT RS256 con Redis en cada operación protegida; rol y recurso según contrato. Rutas internas exigen firma, nonce y autoridad llamante. POST/PATCH de negocio conservan Idempotency-Key donde está declarado, versión y motivo cuando corresponde. GET autorizado genera auditoría; las transacciones publican eventos por outbox.

Contratos: OpenAPI completo · Directorio JSON/XML/XSD · Ejecuciones HTTP.

Método	Ruta	Objetivo	Códigos declarados
POST	/api/v1/shipments	Planear traslado y confirmar vínculo exclusivo con Inventario	202, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/api/v1/shipments	Traslados de la región, institución o responsable	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/api/v1/shipments/{identifier}	Estado logístico y coordinación pendiente	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
POST	/api/v1/shipments/{identifier}/transitions	Registrar paso autorizado y evento de custodia durable	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
POST	/api/v1/shipments/{identifier}/telemetry	Ubicación solo del responsable durante traslado activo	201, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/api/v1/shipments/{identifier}/telemetry	Historial de ubicación con autorización por traslado	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/internal/v1/shipments/{identifier}/access	Autorizar consulta de custodia por asignación logística	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
POST	/api/v1/shipments/{identifier}/files	Preparar carga privada con URL firmada de cinco minutos	201, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/api/v1/shipments/{identifier}/files	Evidencias del traslado y estado persistente de validación	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
POST	/api/v1/files/{identifier}/verify	Verificar tamaño, MIME, firma de archivo y SHA-256 del objeto real	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/api/v1/files/{identifier}/download	Descargar evidencia verificada mediante acceso temporal privado	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/api/v1/summary	Indicadores agregados autorizados; no expone expedientes	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/api/v1/shipments/{identifier}/tracking	Último paso documental con versión y referencia a PostgreSQL	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/api/v1/shipment-statistics	Agregación regional del último paso documental de cada traslado	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503

Entradas y salidas: encabezados Authorization, Accept, Content-Type, X-Correlation-ID y Idempotency-Key según operación. Parámetros de ruta UUID y consulta/paginación constan en OpenAPI. Respuesta {data,meta} o {error,meta}, con representación XML equivalente. El contrato interno añade encabezados de firma del servicio; los valores secretos están redactados en evidencia.

Pruebas: suite de apps/services/tests, expediente dinámico por método/ruta y pruebas Docker de negocio. Consultar RESULTADOS.md para resultados reales y limitaciones; no se considera probado un endpoint solo por aparecer en esta tabla.

Cadena de custodia

Conserva secuencia verificable de transferencias físicas y referencias de evidencia.

Justificación y proceso: Transporte produce PLANNED, PREPARED, COLLECTED, IN_TRANSIT, DELIVERED y ACCEPTED; Custodia los incorpora sin duplicarlos ni admitir huecos.

Integración declarada: puede invocar contratos internos de auditoria, transporte; acepta llamadas firmadas de transporte. Estas relaciones incluyen entrega de eventos. Monitoreo consulta además la salud pública de todos los componentes. Cada ruta conserva su lista específica de consumidores y delegación en OpenAPI.

Persistencia y relaciones: PostgreSQL mantiene el registro autoritativo con secuencia e historial protegido. MongoDB recibe la proyección documental. SHA-256 enlaza eventos; Redis valida identidad y acceso.

Ámbito SQL: access_custodia. La correspondencia siguiente procede del SQL de instalación. Las tablas técnicas service_* filtran el propietario; las claves y relaciones completas constan en el catálogo físico.

Vista autorizada	Tabla física compartida
activity_events	red_house.service_activity_events
chains	red_house.custody_chain
custody_events	red_house.custody_event
decisions	red_house.service_decisions
idempotency	red_house.service_idempotency
inbox	red_house.service_inbox
outbox	red_house.service_outbox
schema_version	red_house.service_schema_version
worker_health	red_house.service_worker_health

Despliegue: Dockerfile, dos instancias custodia-a/b, gateway http://127.0.0.1:6108 y Swagger /docs/. Configuración privada propia: DATABASE_URL, REDIS_URL, claves públicas JWT y de servicios, clave privada de firma interna y URLs del gateway. Solo Identidad conserva la clave privada JWT. Transporte añade configuración GCS; los cuatro propietarios documentales añaden MongoDB.

Seguridad: JWT RS256 con Redis en cada operación protegida; rol y recurso según contrato. Rutas internas exigen firma, nonce y autoridad llamante. POST/PATCH de negocio conservan Idempotency-Key donde está declarado, versión y motivo cuando corresponde. GET autorizado genera auditoría; las transacciones publican eventos por outbox.

Contratos: OpenAPI completo · Directorio JSON/XML/XSD · Ejecuciones HTTP.

Método	Ruta	Objetivo	Códigos declarados
POST	/internal/v1/events	Custodia secuencial, deduplicada e inmutable	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/api/v1/chains/{identifier}	Consultar cadena; hash detecta alteraciones, no certifica veracidad física	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503

Entradas y salidas: encabezados Authorization, Accept, Content-Type, X-Correlation-ID y Idempotency-Key según operación. Parámetros de ruta UUID y consulta/paginación constan en OpenAPI. Respuesta {data,meta} o {error,meta}, con

representación XML equivalente. El contrato interno añade encabezados de firma del servicio; los valores secretos están redactados en evidencia.

Pruebas: suite de apps/services/tests, expediente dinámico por método/ruta y pruebas Docker de negocio. Consultar RESULTADOS.md para resultados reales y limitaciones; no se considera probado un endpoint solo por aparecer en esta tabla.

Alertas

Genera una bandeja de avisos por eventos y conserva su atención.

Justificación y proceso: Una reserva avisa a las instituciones implicadas; una cita o evaluación de donante puede avisar a su cuenta específica.

Integración declarada: puede invocar contratos internos de auditoria; acepta llamadas firmadas de donantes, inventario, monitoreo, receptores, transporte. Estas relaciones incluyen entrega de eventos. Monitoreo consulta además la salud pública de todos los componentes. Cada ruta conserva su lista específica de consumidores y delegación en OpenAPI.

Persistencia y relaciones: PostgreSQL conserva alertas, destinatarios y acuses. Redis valida sesión y permisos. Eventos se deduplican por identificador.

Ámbito SQL: access_alertas. La correspondencia siguiente procede del SQL de instalación. Las tablas técnicas service_* filtran el propietario; las claves y relaciones completas constan en el catálogo físico.

Vista autorizada	Tabla física compartida
activity_events	red_house.service_activity_events
alerts	red_house.notification
decisions	red_house.service_decisions
idempotency	red_house.service_idempotency
inbox	red_house.service_inbox
outbox	red_house.service_outbox
schema_version	red_house.service_schema_version
worker_health	red_house.service_worker_health

Despliegue: Dockerfile, dos instancias alertas-a/b, gateway http://127.0.0.1:6109 y Swagger /docs/. Configuración privada propia: DATABASE_URL, REDIS_URL, claves públicas JWT y de servicios, clave privada de firma interna y URLs del gateway. Solo Identidad conserva la clave privada JWT. Transporte añade configuración GCS; los cuatro propietarios documentales añaden MongoDB.

Seguridad: JWT RS256 con Redis en cada operación protegida; rol y recurso según contrato. Rutas internas exigen firma, nonce y autoridad llamante. POST/PATCH de negocio conservan Idempotency-Key donde está declarado, versión y motivo cuando corresponde. GET autorizado genera auditoría; las transacciones publican eventos por outbox.

Contratos: OpenAPI completo · Directorio JSON/XML/XSD · Ejecuciones HTTP.

Método	Ruta	Objetivo	Códigos declarados
POST	/internal/v1/events	Generar alertas por evento con deduplicación y destinatarios explícitos	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/api/v1/alerts	Bandeja autorizada: institución, rol y usuario	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
POST	/api/v1/alerts/{identifier}/acknowledge	Acusar recibo sin modificar el hecho de origen	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503

Entradas y salidas: encabezados Authorization, Accept, Content-Type, X-Correlation-ID y Idempotency-Key según operación. Parámetros de ruta UUID y consulta/paginación constan en OpenAPI. Respuesta {data,meta} o {error,meta}, con representación XML equivalente. El contrato interno añade encabezados de firma del servicio; los valores secretos están redactados en evidencia.

Pruebas: suite de apps/services/tests, expediente dinámico por método/ruta y pruebas Docker de negocio. Consultar RESULTADOS.md para resultados reales y limitaciones; no se considera probado un endpoint solo por aparecer en esta tabla.

Auditoría

Recibe registros minimizados de accesos, cambios, evaluaciones y decisiones.

Justificación y proceso: El cambio y su evento se guardan juntos en el servicio de origen. Auditoría recibe después el evento con reintentos y deduplicación.

Integración declarada: puede invocar contratos internos de auditoria; acepta llamadas firmadas de alertas, auditoria, compatibilidad, custodia, donantes, identidad_acceso, inventario, monitoreo, priorizacion_geografia, receptores, transporte. Estas relaciones incluyen entrega de eventos. Monitoreo consulta además la salud pública de todos los componentes. Cada ruta conserva su lista específica de consumidores y delegación en OpenAPI.

Persistencia y relaciones: PostgreSQL mantiene registro inmutable bajo credenciales de aplicación. MongoDB proporciona proyección documental flexible. Redis controla sesión, perfil Auditor y región.

Ámbito SQL: access_auditoria. La correspondencia siguiente procede del SQL de instalación. Las tablas técnicas service_* filtran el propietario; las claves y relaciones completas constan en el catálogo físico.

Vista autorizada	Tabla física compartida
activity_events	red_house.service_activity_events
audit_events	red_house.audit_event
decisions	red_house.service_decisions
idempotency	red_house.service_idempotency
inbox	red_house.service_inbox
outbox	red_house.service_outbox
schema_version	red_house.service_schema_version
worker_health	red_house.service_worker_health

Despliegue: Dockerfile, dos instancias auditoria-a/b, gateway http://127.0.0.1:6110 y Swagger /docs/. Configuración privada propia: DATABASE_URL, REDIS_URL, claves públicas JWT y de servicios, clave privada de firma interna y URLs del gateway. Solo Identidad conserva la clave privada JWT. Transporte añade configuración GCS; los cuatro propietarios documentales añaden MongoDB.

Seguridad: JWT RS256 con Redis en cada operación protegida; rol y recurso según contrato. Rutas internas exigen firma, nonce y autoridad llamante. POST/PATCH de negocio conservan Idempotency-Key donde está declarado, versión y motivo cuando corresponde. GET autorizado genera auditoría; las transacciones publican eventos por outbox.

Contratos: OpenAPI completo · Directorio JSON/XML/XSD · Ejecuciones HTTP.

Método	Ruta	Objetivo	Códigos declarados
POST	/internal/v1/events	Recibir registro minimizado y conservarlo de forma inmutable	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/api/v1/events	Bitácora regional de solo lectura; sin credenciales ni contenido clínico	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/api/v1/events/{identifier}	Evento auditable y correlación	200, 400, 401, 403, 404, 406, 409,

Método	Ruta	Objetivo	Códigos declarados
			413, 415, 422, 429, 500, 503
GET	/api/v1/documentary-events	Proyección documental MongoDB; puede estar pendiente de sincronización	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503

Entradas y salidas: encabezados Authorization, Accept, Content-Type, X-Correlation-ID y Idempotency-Key según operación. Parámetros de ruta UUID y consulta/paginación constan en OpenAPI. Respuesta {data,meta} o {error,meta}, con representación XML equivalente. El contrato interno añade encabezados de firma del servicio; los valores secretos están redactados en evidencia.

Pruebas: suite de apps/services/tests, expediente dinámico por método/ruta y pruebas Docker de negocio. Consultar RESULTADOS.md para resultados reales y limitaciones; no se considera probado un endpoint solo por aparecer en esta tabla.

Monitoreo

Consulta salud y conserva disponibilidad, latencia, versión y dependencia que falla.

Justificación y proceso: Con Redis detenido, el proceso sigue vivo, readiness se degrada y las operaciones protegidas fallan de forma cerrada.

Integración declarada: puede invocar contratos internos de alertas, auditoria; no ofrece contratos internos firmados. Estas relaciones incluyen entrega de eventos. Monitoreo consulta además la salud pública de todos los componentes. Cada ruta conserva su lista específica de consumidores y delegación en OpenAPI.

Persistencia y relaciones: HTTP requests consulta /health/ready; MongoDB guarda primero cada sondeo y PostgreSQL conserva las verificaciones. El estado puede consultarse desde MongoDB si PostgreSQL falla. Redis protege las consultas. El worker sondea cada 30 segundos.

Ámbito SQL: access_monitoreo. La correspondencia siguiente procede del SQL de instalación. Las tablas técnicas service_* filtran el propietario; las claves y relaciones completas constan en el catálogo físico.

Vista autorizada	Tabla física compartida
activity_events	red_house.service_activity_events
checks	red_house.service_health_check
decisions	red_house.service_decisions
idempotency	red_house.service_idempotency
inbox	red_house.service_inbox
outbox	red_house.service_outbox
schema_version	red_house.service_schema_version
worker_health	red_house.service_worker_health

Despliegue: Dockerfile, dos instancias monitoreo-a/b, gateway http://127.0.0.1:6111 y Swagger /docs/. Configuración privada propia: DATABASE_URL, REDIS_URL, claves públicas JWT y de servicios, clave privada de firma interna y URLs del gateway. Solo Identidad conserva la clave privada JWT. Transporte añade configuración GCS; los cuatro propietarios documentales añaden MongoDB.

Seguridad: JWT RS256 con Redis en cada operación protegida; rol y recurso según contrato. Rutas internas exigen firma, nonce y autoridad llamante. POST/PATCH de negocio conservan Idempotency-Key donde está declarado, versión y motivo cuando corresponde. GET autorizado genera auditoría; las transacciones publican eventos por outbox.

Contratos: OpenAPI completo · Directorio JSON/XML/XSD · Ejecuciones HTTP.

Método	Ruta	Objetivo	Códigos declarados
GET	/api/v1/status	Estado, promedio, último error, versión e historial resumido	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503
GET	/api/v1/checks	Historial persistente de verificaciones	200, 400, 401, 403, 404, 406, 409, 413, 415, 422, 429, 500, 503

Entradas y salidas: encabezados Authorization, Accept, Content-Type, X-Correlation-ID y Idempotency-Key según operación. Parámetros de ruta UUID y consulta/paginación constan en OpenAPI. Respuesta {data,meta} o {error,meta}, con representación XML equivalente. El contrato interno añade encabezados de firma del servicio; los valores secretos están redactados en evidencia.

Pruebas: suite de apps/services/tests, expediente dinámico por método/ruta y pruebas Docker de negocio. Consultar RESULTADOS.md para resultados reales y limitaciones; no se considera probado un endpoint solo por aparecer en esta tabla.

Referencias y archivos de esta sección

Directorio JSON/XML/XSD: apps/services/ms_alertas/contracts

Directorio JSON/XML/XSD: apps/services/ms_auditoria/contracts

Directorio JSON/XML/XSD: apps/services/ms_compatibilidad/contracts

Directorio JSON/XML/XSD: apps/services/ms_custodia/contracts

Directorio JSON/XML/XSD: apps/services/ms_donantes/contracts

Directorio JSON/XML/XSD: apps/services/ms_identidad_acceso/contracts

Directorio JSON/XML/XSD: apps/services/ms_inventario/contracts

Directorio JSON/XML/XSD: apps/services/ms_priorizacion_geografia/contracts

Directorio JSON/XML/XSD: apps/services/ms_receptores/contracts

Directorio JSON/XML/XSD: apps/services/ms_transporte/contracts

Directorio JSON/XML/XSD: apps/services/servicio_monitoreo/contracts

Dockerfile: apps/services/ms_alertas/Dockerfile

Dockerfile: apps/services/ms_auditoria/Dockerfile

Dockerfile: apps/services/ms_compatibilidad/Dockerfile

Dockerfile: apps/services/ms_custodia/Dockerfile

Dockerfile: apps/services/ms_donantes/Dockerfile

Dockerfile: apps/services/ms_identidad_acceso/Dockerfile

Dockerfile: apps/services/ms_inventario/Dockerfile

Dockerfile: apps/services/ms_priorizacion_geografia/Dockerfile

Dockerfile: apps/services/ms_receptores/Dockerfile

Dockerfile: apps/services/ms_transporte/Dockerfile

Dockerfile: apps/services/servicio_monitoreo/Dockerfile

Ejecuciones HTTP: documentation/evidencias/segundo-parcial/http/index.html

OpenAPI completo: apps/services/ms_alertas/contracts/openapi.json

OpenAPI completo: apps/services/ms_auditoria/contracts/openapi.json

OpenAPI completo: apps/services/ms_compatibilidad/contracts/openapi.json

OpenAPI completo: apps/services/ms_custodia/contracts/openapi.json

OpenAPI completo: apps/services/ms_donantes/contracts/openapi.json

OpenAPI completo: apps/services/ms_identidad_acceso/contracts/openapi.json

OpenAPI completo: apps/services/ms_inventario/contracts/openapi.json

OpenAPI completo: apps/services/ms_priorizacion_geografia/contracts/openapi.json

OpenAPI completo: apps/services/ms_receptores/contracts/openapi.json

OpenAPI completo: apps/services/ms_transporte/contracts/openapi.json

OpenAPI completo: apps/services/servicio_monitoreo/contracts/openapi.json

catálogo físico: documentation/segundo-parcial/Modelo_fisico.md

Matriz de ejecución del segundo parcial

Inicio: 15/10/2026. Entrega: 22/10/2026. Fuente: enunciado externo y stuff/prompts/completar_segundo_parcial.md. El árbol inicial estaba limpio.
Las pruebas del 14 y 15 de octubre son antecedentes, no resultados de este cambio.

Requisito	Estado inicial comprobado	Cambio entregado	Dependencias	Criterio y evidencia final
Continuidad web	Portal exclusivamente HTTP	Backend modular propio, reglas versionadas y workers autónomos	PostgreSQL compartido, Redis y módulos autorizados	Aprobado: sangre y órganos con las 22 API detenidas; Docker final; donación/procesamiento en la regresión
Disponibilidad JSON/XML	Una instancia por API	Dos instancias de los once propietarios, incluida Identidad, más workers y DNS	Gateway y contratos compatibles	Aprobado: once réplicas A detenidas, flujo de órganos y consultas JSON/XML; actualización real de Inventario, 100 consultas 200 y altas persistidas
PostgreSQL compartido	Base red_house ya migrada	Se conservan 71 tablas, permisos por servicio y vistas access_*	Esquema versión 6 y credenciales separadas	Aprobado: regresión, catálogo físico de 145 índices y restauración completa; entorno habitual conservado
Docker	Sin Dockerfile/Compose en el inicio del incremento	Imágenes por propietario/web, redes, secretos, supervisión y volúmenes	Docker Compose y acceso a registro de imágenes	Aprobado: 27 contenedores saludables, continuidad, actualización y reinicio; clon limpio con ocho etapas, negocio real y bootstrap repetido
MongoDB	Eventos inmutables y pipelines de Monitoreo existentes	Proyección mutable de seguimiento, índices y reconstrucción	Roles documentales limitados y eventos SQL relacionados	Aprobado: versión, duplicados, desorden, reconstrucción, filtrado, pipelines e índice IXSCAN; resultado
Redis	Sesiones, permisos y caché existentes	Participación común de monolito y réplicas, TTL y revocación conservados	ACL y claves por prefijo	Aprobado: login/rotación/revocación, límites, nonce, caché y rechazo durante caída en regresión; evidencia SQL/Redis por endpoint
GCS	Adaptador sin bucket ni interfaz completa	Flujo web privado, validación, estados y descarga firmada	Recursos/IAM autorizados y ADC impersonada	Aprobado: carga/descarga/hash, permiso público denegado, inválido, ausente, expirado y recuperación; GCS real, navegador
Web y seguridad	Procesos existentes y	Integración autónoma,	Roles institucionales, JWT y	Aprobado: 182 pruebas

Requisito	Estado inicial comprobado	Cambio entregado	Dependencias	Criterio y evidencia final
	resultados históricos	gestión de archivos y configuración privada corregida	Redis	finales y 20 páginas de siete perfiles; Highcharts con datos comprobados, filtros/paginación y ancho 390
JSON/XML/XSD e internas	Contratos versionados existentes	Contratos actualizados, ejemplos correctos/error, archivo/proyección y evidencia real	Validadores, firmas Ed25519 y JWT delegado	Aprobado: regresión, consultas Docker y expediente HTTP; Swagger ejecutado
Evidencia por endpoint	Ejemplos sin expediente exhaustivo	Inventario dinámico, registros redactados, persistencia y capturas	Pruebas finales y visor local	Aprobado: 284/284 éxitos HTTP y capturas; 240 con error real; 44 GET de documentación fija con no aplicabilidad justificada; cobertura
Fallos y recuperación	Pruebas históricas	Nuevas comprobaciones de dependencias, instancia, monolito y actualización	Entornos aislados	Aprobado: nueve controles Docker y una prueba complementaria de salud para los once propietarios; incidentes conservados
Respaldo	Procedimiento sin ensayo integral de este despliegue	Exportación coordinada y restauración en tres contenedores nuevos	Aplicaciones pausadas, secretos privados y volúmenes nuevos	Aprobado: 71 tablas/15 122 filas SQL, 7 632 documentos MongoDB, usuarios/roles por hash y clave Redis con TTL; comparación
Documento técnico	Guías dispersas y revisión histórica	Informe, once fichas, diagramas, catálogo físico y trazabilidad	Código y resultados fechados	Entregado: informe Word / PDF, modelo físico, once fichas y figuras actuales
Presentación	Apuntes individuales	Guion presencial y de video, reparto por responsabilidad y demo reproducible	Participación y publicación reales del equipo	Material preparado; requisito externo pendiente: ensayo, presentación, grabación, publicación y enlace de YouTube

Decisión de continuidad

El monolito ejecuta dentro de su proceso los módulos de aplicación distribuidos también en las imágenes de cada servicio. El adaptador WSGI conserva validación, JWT, autorización Redis, firmas internas, transacciones e idempotencia. No inicia servidores HTTP internos ni cambia de backend tras un error. Cada módulo usa credenciales limitadas a sus vistas del esquema compartido. La entrega fija el código con versión y manifiesto; actualizar una API no cambia el monolito en uso.

Los workers propios del monolito entregan el outbox mediante el mismo adaptador. Los workers de las API usan enrutamiento HTTP. Comparten colas con bloqueo SQL y consumidores idempotentes. No hay doble escritura de entidades ni copia de reglas.

Fallo	Web modular	Clientes JSON/XML	Límite
Una instancia API o worker	Opera con módulos y workers propios	Enruta a instancia restante	Respuesta perdida tras escritura exige consultar operación/idempotencia
Todas las instancias de un propietario API	Opera mientras sus dependencias de datos estén disponibles	Operaciones dependientes fallan controladamente	No se afirma continuidad API sin autoridad disponible

Fallo	Web modular	Clientes JSON/XML	Límite
Redis	Rechaza operaciones protegidas	Rechaza operaciones protegidas	No se omite autenticación
PostgreSQL	Operaciones transaccionales indisponibles	Operaciones transaccionales indisponibles	No hay réplica de infraestructura en este alcance
MongoDB	SQL conserva outbox; consulta documental degradada	Mismo comportamiento	Recuperación y deduplicación posterior
GCS	Operaciones sin archivos continúan; archivo pendiente/error	Mismo comportamiento	No se declara verificado un objeto no comprobado

Recursos externos autorizados

Proyecto sistemas-integracion-01; bucket privado red-house-evidencias-sistemas-integracion-01, us-central1, Standard; acceso uniforme y prevención de acceso público. Cuenta red-house-archivos, permisos de objetos limitados al bucket y firma limitada a esa cuenta. Sin claves privadas versionadas. Configuración autorizada expresamente por el usuario durante esta ejecución.

Evidencia y límites del cierre

La regresión final aprobó 182 pruebas en 2005.90 s, sin fallos ni omisiones. La prueba complementaria de salud aprobó después en 36.08 s; no se suma a esa ejecución como si ambas fueran una sola suite. Los resultados detallan esperado, obtenido, fallos y correcciones.

La actualización Docker sustituyó imágenes compatibles de Inventario A/B; no prueba compatibilidad con migraciones futuras desconocidas. La restauración de las 22 API incluyó suspensión por cierre de tapa, documentada por separado; su tiempo de pared no acredita RTO. No se ampliaron deadlines para obtener aprobaciones.

La base, Redis, MongoDB, gateway y host no tienen redundancia de infraestructura en este alcance. GCE, HTTPS público, operación clínica y capacidad productiva no quedan certificados. El respaldo local no copia objetos GCS: se deben conservar las generaciones referenciadas en el bucket.

Recuperación automatizada de contraseña, email/push, agenda detallada por sede/hora y reportes regulatorios certificados conservan justificación como pendientes de alcance posterior. La ayuda de acceso no recupera contraseñas; CSV de la página actual no equivale a exportación completa ni reporte regulatorio. Android y escritorio están excluidos expresamente del segundo parcial; sus contratos JSON/XML sí se prueban.

Las observaciones textuales del docente no fueron proporcionadas. El informe describe correcciones técnicas verificadas sin inventar su procedencia. Los nombres y la portada se omiten por instrucción del usuario. El equipo agregará los datos de identificación y la participación reales cuando corresponda.

La entrega académica completa sigue pendiente de presentación presencial y YouTube; no se atribuyen al código esos requisitos externos.

Actualización de sesiones del 20 de octubre

El portal conserva credenciales cifradas y concesiones de renovación en Redis. Los cierres y revocaciones pendientes quedan en PostgreSQL, esquema web_runtime, con credenciales limitadas. Se mantienen las 71 tablas del dominio y se añade una tabla técnica; 72 tablas entre ambos esquemas. La validación posterior registra migración, concurrencia, caídas, recuperación, HTTP Docker y restauración. Las pruebas del cierre anterior mantienen su alcance y fecha.

Referencias y archivos de esta sección

Docker final: documentation/evidencias/segundo-parcial/docker-release/docker-results.json

GCS real: documentation/evidencias/segundo-parcial/gcs-real.json

Guion presencial y de video: stuff/presentacion/Guion_segundo_parcial.md

PDF: documentation/pdf/Reporte_tecnico_segundo_parcial.pdf

clon limpio: documentation/evidencias/segundo-parcial/clean-clone.json

cobertura: documentation/evidencias/segundo-parcial/http/coverage.json

comparación: documentation/evidencias/segundo-parcial/backup-restore.json

expediente HTTP: documentation/evidencias/segundo-parcial/http/index.html

informe Word: documentation/docx/Reporte_tecnico_segundo_parcial.docx

modelo físico: documentation/segundo-parcial/Modelo_fisico.md

prueba complementaria de salud: documentation/evidencias/segundo-parcial/salud-complementaria.xml

regresión final: documentation/evidencias/segundo-parcial/regresion-final.xml

resultado: documentation/evidencias/segundo-parcial/mongo-projection.json

resultados: documentation/evidencias/segundo-parcial/RESULTADOS.md

validación posterior: documentation/evidencias/sesiones-redis/RESULTADOS.md

Resultados del segundo parcial

Validación del 19 de octubre de 2026. Aplicación 0.3.0. La regresión, continuidad Docker, navegador, restauración de respaldos e instalación desde clon limpio están aprobados. Los artefactos técnicos se entregan sin portada ni nombres. Presentación presencial y YouTube corresponden al equipo.

Condiciones y datos

macOS, Python 3.12 local, Docker Desktop con 8 CPU y 7.75 GiB asignados; imágenes Python 3.13, PostgreSQL 14, Redis 7.4 y MongoDB 8. Datos ficticios y entornos aislados. El entorno habitual .local/runtime no se utilizó para las interrupciones. Bucket GCS privado real autorizado; credenciales fuera del expediente.

Resultado esperado y obtenido

Criterio	Esperado	Obtenido	Evidencia
Regresión final	Flujos, seguridad, contratos, concurrencia y persistencia correctos	182 aprobadas, cero fallos y cero omisiones; 2005.90 s	JUnit, salida
Salud complementaria	Negociación inválida 406; dependencia inaccesible 503 y recuperación 200	Una prueba adicional aprobada, 36.08 s, para los once propietarios	JUnit, salida
Sangre por API Docker	Reserva única, traslado, solicitud cerrada y recurso recibido	Aprobado con comprobación SQL y custodia	Nueve controles Docker
Órganos y JSON/XML sin réplicas A	Flujo completo y consultas de los once servicios mediante B, incluida Identidad	Aprobado	Intercambios Docker
Web sin las 22 API	Sangre y órganos de principio a fin mediante backend propio	Ambos aprobados; restauración posterior completada	Docker final
Todas las autoridades API detenidas	Informar indisponibilidad sin eludir seguridad	Gateway 502	Docker final
Actualización gradual	Sustituir A y B por imagen compatible conservando operaciones	IDs de imagen distintos; 100 consultas continuas, todas 200; alta y consulta SQL después de cada sustitución	Imágenes y resultados
Reinicio de almacenes	Conservar datos y recuperar consultas autorizadas	Mismo ID, folio, estado y versión SQL; JSON/XML recuperados tras reiniciar PostgreSQL, Redis y MongoDB	Comparación antes/después

Criterio	Esperado	Obtenido	Evidencia
GCS real	Carga privada, descarga/hash, permisos, validación y recuperación	Web 303; descarga 200/hash idéntico; anónimo 403; inválido 422; ausente 404; expirado 400 ExpiredToken; dependencia 503 y recuperación 200	Resultado GCS, configuración de bucket
Proyección MongoDB	Actualizar sin modificar eventos, deduplicar, ordenar, reconstruir e indexar	Versión 3; seis eventos reproducidos; duplicados/reconstrucción sin cambios; actualización de evento denegada; IXSCAN regional_state, tres claves/documentos examinados	Resultado y explain
Navegador	Páginas autorizadas, gráficas reales, adaptación móvil, archivo y Swagger	20 páginas, siete perfiles, sin errores JavaScript; valores y etiquetas Highcharts coinciden; ancho 390 sin desbordamiento; carga VERIFIED y Swagger 200	Informe, archivo, Swagger ejecutado
HTTP por endpoint	Solicitudes reales, errores aplicables, persistencia y captura	284/284 con HTTP exitoso; 240 con error; 44 rutas de documentación fija con justificación de no aplicabilidad; ninguna escritura capturada sin comprobación	Visor, cobertura, 284 capturas de endpoints y una del inventario
Respaldo y restauración aislada	Igualdad de datos, roles y estado temporal	71 tablas / 15 122 filas SQL, 7 632 documentos MongoDB, cinco usuarios y cuatro roles documentales idénticos por conteo/hash; clave Redis restaurada con TTL 3575 s	Comparación completa
Instalación desde clon limpio	Secretos nuevos, esquema y volúmenes propios, arranque, negocio y bootstrap repetible	Ocho etapas aprobadas; 27 contenedores saludables; sangre por API, consultas JSON/XML de once propietarios y segunda ejecución de bootstrap	Informe, estado, HTTP real

Los 182 casos pertenecen a una sola ejecución completa. La prueba de salud se agregó y ejecutó después, sin cambiar código de aplicación; se informa por separado. Las nueve comprobaciones Docker y los recorridos de navegador tampoco se suman como casos de esa regresión.

La prueba complementaria de salud dirige temporalmente cada dependencia a un puerto local cerrado. Los handlers intentan conexiones reales y se verifican sus respuestas HTTP y su recuperación; no se fabrican respuestas del adaptador. Los ensayos de reinicio Docker sí reinician los motores completos.

Fallos conservados y correcciones

- La primera suite completa del 19/10 obtuvo 176 aprobados y un fallo GCS: autenticación/firma podían esperar sin límite explícito. Se acotaron esos RPC; la regresión final con GCS aprobó. No se amplió el timeout de las pruebas.
- Una aserción inicial esperaba 403 para URL expirada. GCS devuelve 400 ExpiredToken; se ajustó al contrato documentado manteniendo la comprobación exacta del código XML. Contrato XML GCS.
- La captura optativa de archivos estáticos intentaba consumir send_file en modo streaming. Ahora materializa la respuesta solo cuando se solicita evidencia; la ejecución HTTP de todas las rutas pasó.
- Las primeras pruebas Docker encontraron directorios temporales Nginx incompatibles con raíz de solo lectura y carga tardía de configuración web privada. Se corrigieron y se repitieron los flujos.
- Las conexiones SQL costosas bajo carga motivaron pools acotados por aplicación, credencial y esquema. La regresión verifica reutilización, rollback, aislamiento y recuperación sin repetición automática de escrituras. Pool Psycopg.
- Un primer 504 durante la caída de réplicas permanece registrado. No se atribuye una causa única sin evidencia suficiente. Las repeticiones y el ensayo final completaron los procesos previstos.

- Restaurar muchas instancias simultáneamente agotó salud bajo carga del host superior a 300. Se ordenó el arranque por pares, consumidores primero. El sondeo mínimo conservó verificación HTTP y del worker, sin ampliar los 8 s del health check. La ejecución final completó ambas restauraciones.
- La restauración final de las 22 API incluyó suspensión del equipo: cierre de tapa a las 11:24:40 y apertura a las 12:16:02. El registro de energía distingue esa pausa; la duración de pared de la fase no mide RTO ni latencia. Los escenarios de negocio anteriores y actualización/reinicio posteriores terminaron con el equipo despierto.
- El capturador de navegador primero consultó una columna inexistente y después eligió un traslado cerrado. Se corrigió la consulta y se preparó un traslado abierto mediante API. La aplicación correctamente impide cargar nuevas evidencias fuera del periodo autorizado. Incidente y repetición.
- El ensayo de respaldo detectó escapado incorrecto del script embebido, pérdida de autenticación al restaurar MongoDB admin junto con índices, y orden variable en el agregado SQL. La restauración ahora separa credenciales y datos con nueva autenticación; compara SQL ordenado por tabla y verifica hashes de usuarios/roles. Incidentes, comportamiento mongorestore.
- La primera selección del clon rechazó el marcador versionado .codegraph/.gitignore. Se permitió únicamente ese archivo de configuración, conservando el rechazo de secretos y cachés. Diagnóstico.
- El primer intento de instalación del clon quedó detenido al fallar DNS de Docker Hub. Se comprobó después la resolución desde host y contenedor; otra instalación completa, con proyecto y volúmenes nuevos, aprobó. Fallo conservado, repetición aprobada.

Los metadatos de regresiones intermedias conservan fechas, hashes y fallos anteriores. Los informes Docker intermedios permanecen en sus carpetas; el cierre usa docker-release, no borra ni reetiqueta sus predecesores.

Alcance y límites de la evidencia

El inventario se obtiene de rutas reales y excluye los métodos automáticos HEAD/OPTIONS. Cada método y ruta tiene su propia página, éxito, error cuando aplica, headers redactados, fecha, versión, correlación y huella de implementación. Los cuerpos completos quedan enlazados. Las capturas del visor corresponden a intercambios ejecutados; los ejemplos Swagger se distinguen de esa evidencia. El expediente separa HTTP, WSGI y llamadas modulares.

Las escrituras conservan consultas posteriores a idempotencia, operación, inbox o sesión SQL/Redis. Recibir un 200 por sí solo no se usa como prueba de persistencia. Las 44 rutas GET de documentación fija no reciben datos de negocio; se explica por qué no se fabrica un error de dominio para ellas.

La restauración compara datos con las aplicaciones pausadas y usa tres contenedores nuevos sin puertos publicados. Los archivos privados de respaldo incluyen configuración y claves. El objeto ficticio del navegador permanece en el bucket para demostración; el respaldo local no copia objetos GCS y su recuperación requiere conservar las generaciones referenciadas. Los objetos creados por la prueba automatizada GCS se eliminaron en su limpieza.

No se certifican carga productiva, migraciones futuras incompatibles, GCE/HTTPS público, alta disponibilidad de almacenes/host, validez clínica, presentación presencial ni publicación de YouTube. Las mediciones históricas de carga se conservan con su fecha y no se atribuyen a 0.3.0.

Estado al entregar

Los contenedores de demostración, restauración y clones de prueba quedan detenidos; sus volúmenes y respaldos privados se conservan. Para recuperar el Compose principal desde la raíz: python3 deployment/start.py. El entorno habitual .local/runtime se mantiene separado. Los 630 archivos de aplicación/despliegue comparados coinciden con la instantánea del clon; los hashes de los once servicios coinciden con la regresión final. Verificación de fuentes.

Referencias y archivos de esta sección

Comparación antes/después: documentation/evidencias/segundo-parcial/docker-release/docker-results.json

Comparación completa: documentation/evidencias/segundo-parcial/backup-restore.json

Contrato XML GCS: https://docs.cloud.google.com/storage/docs/xml-api/reference-status

Diagnóstico: documentation/evidencias/segundo-parcial/clean-clone-incidents.json

Docker final: documentation/evidencias/segundo-parcial/docker-release/docker-results.json

Fallo conservado: documentation/evidencias/segundo-parcial/clean-clone-dns-failure.json

HTTP real: documentation/evidencias/segundo-parcial/clean-clone-http.json

Imágenes y resultados: documentation/evidencias/segundo-parcial/docker-release/docker-results.json

Incidente y repetición: documentation/evidencias/segundo-parcial/browser-incidents.json

Incidentes: documentation/evidencias/segundo-parcial/backup-incidents.json

Informe: documentation/evidencias/segundo-parcial/browser/browser-results.json

Informe: documentation/evidencias/segundo-parcial/clean-clone.json

Intercambios Docker: documentation/evidencias/segundo-parcial/docker-release/docker-http.json

JUnit: documentation/evidencias/segundo-parcial/regresion-final.xml

JUnit: documentation/evidencias/segundo-parcial/salud-complementaria.xml

Nueve controles Docker: documentation/evidencias/segundo-parcial/docker-release/docker-results.json

Pool Psycopg: https://www.psycopg.org/psycopg3/docs/advanced/pool.html

Resultado GCS: documentation/evidencias/segundo-parcial/gcs-real.json

Resultado y explain: documentation/evidencias/segundo-parcial/mongo-projection.json

Swagger ejecutado: documentation/evidencias/segundo-parcial/browser/swagger-executed.png

Verificación de fuentes: documentation/evidencias/segundo-parcial/source-validation.json

Visor: documentation/evidencias/segundo-parcial/http/index.html

archivo: documentation/evidencias/segundo-parcial/browser/gcs-web-verified.png

cobertura: documentation/evidencias/segundo-parcial/http/coverage.json

comportamiento mongorestore: https://www.mongodb.com/docs/database-tools/mongorestore/

configuración de bucket: documentation/evidencias/segundo-parcial/gcs-bucket.json

estado: documentation/evidencias/segundo-parcial/clean-clone-start.json

metadatos de regresiones intermedias: documentation/evidencias/segundo-parcial/regresiones-intermedias.json

registro de energía: documentation/evidencias/segundo-parcial/docker-release/host-suspension.json

repetición aprobada: documentation/evidencias/segundo-parcial/clean-clone.json

salida: documentation/evidencias/segundo-parcial/regresion-final.txt

salida: documentation/evidencias/segundo-parcial/salud-complementaria.txt

Validación de sesiones Redis — 20 de octubre de 2026

La aplicación utiliza Redis para sesiones técnicas cifradas y concesiones de renovación. PostgreSQL conserva cierres y revocaciones pendientes en web_runtime.session_revocations, con credenciales exclusivas. El navegador recibe una cookie opaca. Los archivos del almacén anterior permanecen en respaldos privados; la aplicación ya no los abre ni los monta.

Criterios de aceptación y resultados

Comprobación	Resultado observado	Evidencia
Regresión de web, continuidad, seguridad, instalador y contratos	121 pruebas aprobadas, sin fallos, errores ni omisiones; 798.33 s	JUnit, salida
Validación final del módulo, incluido descifrado fallido	9 pruebas aprobadas; 82.26 s. Ejecución posterior al endurecimiento del manejo de	JUnit, salida

Comprobación	Resultado observado	Evidencia
	credenciales indescifrables	
HTTP real en Docker, backend autónomo y todas las API detenidas	Login 302, renovación 200, logout 302; cookie cerrada redirige a login. Tokens cifrados, TTL 86380 s; sin volumen de sesiones local	Docker
Migración local y Docker repetible	120 681 filas locales y 15 122 Docker conservadas, conteos iguales en cada una de las 71 tablas del dominio; credenciales estables al repetir bootstrap	Migración
Respaldo y restauración en motores nuevos	15209 filas de 72 tablas idénticas por conteo/hash, incluida la cola técnica; rol del portal con acceso técnico y sin acceso al esquema de dominio. Redis restaurado, TTL de prueba 3531 s	Restauración

Las nueve pruebas finales verifican cifrado y expiración, lectura desde dos instancias web, ACL Redis y permisos SQL, renovación simultánea, logout durante renovación, caída de Redis y recuperación, caída de PostgreSQL, repetición de la actualización y recuperación de una clave de cifrado. No se suman las dos ejecuciones como pruebas distintas: ocho casos del módulo ya están incluidos en la regresión.

Comportamiento ante fallos

Las sesiones duran hasta 24 horas; renovar no amplía ese límite. Las concesiones duran 60 segundos y verifican su propietario antes de guardar o liberar. Una renovación concurrente rechazada devuelve 409. La marca PostgreSQL evita reactivar una cookie cerrada desde una copia antigua de Redis.

Si Redis falla durante logout, el cierre queda registrado en PostgreSQL y el worker completa la revocación cuando vuelve Redis. Si PostgreSQL falla, el acceso protegido devuelve 503; logout elimina la cookie, pero responde 503 al no poder confirmar el cierre global. Una credencial indescifrable permanece pendiente, sin marcarse como revocada. Las marcas vencidas se eliminan; las credenciales confirmadas se borran del registro inmediatamente.

Reproducción y alcance

```
apps/services/.venv/bin/python -m pytest apps/services/tests/test_web_sessions.py -q
apps/services/.venv/bin/python deployment/verify_web_sessions.py
apps/services/.venv/bin/python deployment/backup.py --verify --leave-stopped --output
documentation/evidencias/sesiones-redis/backup-restore.json
```

La prueba pytest usa almacenes temporales independientes. Los comandos Docker requieren una instalación preparada; la verificación HTTP requiere el portal iniciado. El respaldo conserva datos y secretos fuera del repositorio. Las pruebas de caídas usan conexiones a un puerto cerrado y las pruebas de continuidad detienen listeners de API. La restauración usa motores nuevos. No se presenta como prueba de carga sostenida ni de disponibilidad entre hosts.

Las sesiones anteriores ya estaban vencidas: no había sesiones ni revocaciones vigentes que importar. Los usuarios deben iniciar sesión nuevamente. Las guías de instalación local y Docker describen la actualización.

Los incidentes registran un primer fallo por escritura exclusiva de configuración y un error posterior de descubrimiento de fixtures. Ambos fueron corregidos antes de las ejecuciones aprobadas. Las evidencias del 19 de octubre se conservan con su fecha y alcance originales.

Referencias y archivos de esta sección

Docker: deployment/README.md

Docker: documentation/evidencias/sesiones-redis/docker.json

JUnit: documentation/evidencias/sesiones-redis/regresion.xml

JUnit: documentation/evidencias/sesiones-redis/sesiones-final.xml

Migración: documentation/evidencias/sesiones-redis/migracion.json

Restauración: documentation/evidencias/sesiones-redis/backup-restore.json

incidentes: documentation/evidencias/sesiones-redis/incidentes.json

instalación local: documentation/markdowns/Instalacion_y_operacion.md

salida: documentation/evidencias/sesiones-redis/regresion.txt

salida: documentation/evidencias/sesiones-redis/sesiones-final.txt